Bulgarian Diploma Thesis

DDoS Detection under Network Address Translation: A Comparative Study of One-Class SVM and k-Nearest Neighbors

Nikola Georgiev

ID: 200191333

Student: Nikola Georgiev

Date: 05.12.2025

Supervisor: Zachary Hutchinson

Date: 05.12.2025

Department of Computer Science, AUBG

Blagoevgrad, 2025

Contents

1 Introduction **1**

1.1 Related Work 1

1.2 Background and Context 3

1.3 Problem Statement 3

1.4 Objectives 4

1.5 Contributions 4

1.6 Thesis Structure 5

2 Specification of the Software Requirements and their Analysis **6**

2.1 Requirements and their Analysis 6

2.1.1 Start/Stop Capture 6

2.1.2 Feature Computation 7

2.1.3 Feature Delivery API 8

2.1.4 Analyze on Demand 8

2.1.5 Continuous Monitoring 10

2.1.6 Get Current State 10

2.1.7 Model Management 10

2.1.8 Data Scaling 12

2.2 Constraints 13

2.2.1 Microservice Containerization 13

3 Design of the Software Solution **14**

3.1 Project Architecture Overview 14

3.2 Class Architecture 18
3.3 Implemented Algorithms 19
3.3.1 One-Class Support Vector Machines (OC-SVM) 20
3.3.2 Kernel Functions Used for OC-SVM 22
3.3.3 k-Nearest Neighbor (KNN) 27
3.3.4 Principal Component Analysis 29

4 Implementation 32
4.1 Technologies and Implementation Approach 32
4.2 Programming Languages, Libraries, and Frameworks 33
4.3 Installation Guidelines 36
4.4 Illustrative Code Listings 37
4.4.1 Pybind11 Module Definition 37
4.4.2 CUDA Kernel for Pairwise Distance 38
4.4.3 FastAPI Endpoint: `/estimate_attack` 39

5 Testing 42
5.1 Experimental Protocol 43
5.2 Types of DDoS Attacks 44
5.2.1 SYN Flood Attacks 44
5.2.2 HTTP Flood Attacks 45
5.3 Unit Tests 46
5.4 Evaluation Metrics 47
5.4.1 Supporting Metrics 48

6 Results and Conclusion 51
6.1 Detailed Analysis of the Contaminated-Data Failure Mode 53
6.2 Limitations 54
6.3 Future Work 54

References 56

List of Figures

1.1 Network architecture overview. 5

2.1 Feature Delivery Process. 9

2.2 Get Current state Overview. 11

3.1 Micro services Architecture Overview. 15

3.2 UML Class Diagram. 18

3.3 Oc-SVM Classification. 21

3.4 Linear Kernel Function. 23

3.5 Poly Kernel. 24

3.6 Radial Basis Function: 2D -> Hyper Space. 25

3.7 Sigmoid kernel classification. 26

3.8 KNN Classification. 28

4.1 Connection between C++ algorithms Library and python micro services. 36

5.1 GNS3 simulation of a enterprise network. 42

5.2 SYN Flood. 45

5.3 HTTP Flood. 45

5.4 Unit Test. 46

5.5 ROC-AUC. 48

5.6 Recall. 49

List of Tables

3.1 Comparative properties of the four OC-SVM kernel functions. 26

5.1 Confusion matrix conventions used throughout this work. 49

6.1 Results from testing loaded models. 51

Chapter 1

Introduction

Distributed Denial of Service (DDoS) attacks are one of the most severe and persistent threats to the stability, reliability, and availability of online systems. These attacks overwhelm network infrastructure by flooding targeted servers or services with excessive traffic, resulting in significant service degradation or complete unavailability. As a System Engineer at A1 Telekom Bulgaria, I witnessed firsthand how these attacks can degrade services and make large-scale network infrastructures unusable. This project was conceived as a practical and technical response to that problem — using data analysis and classification techniques to identify and mitigate DDoS attacks in real time.

The project focuses on the development of a C++ library designed for DDoS detection and classification. The library is then used with a two-service FastAPI microservice architecture, consisting of two primary components: one that is responsible for data collection and a second that is for network analysis.

The motivation behind this project stems from real-world challenges encountered in telecommunications infrastructure. DDoS attacks are not only increasing in frequency but also evolving in sophistication. Attackers often use botnets of compromised IoT devices or amplification techniques to launch multi-vector attacks that are difficult to detect through traditional signature-based systems.

As a person with both networking and software development knowledge, I can propose a solution to the community that is easy to integrate with their current systems in order to detect ongoing malicious activities.

1.1 Related Work

The literature on machine-learning-based DDoS detection has expanded substantially over the past five years, driven by both the growing severity of the threat and the matu-

ration of the underlying tooling. For the purposes of this thesis, the most relevant prior art clusters into three groups.

The first group applies *supervised* classifiers to labeled attack corpora. Representative examples include the random-forest approach of Alduailij et al. [2], the mutual-information feature-selection pipeline of Abiramasundari and Ramaswamy [1], and the deep-learning variants surveyed by Arango-López et al. [4]. These methods consistently achieve near-perfect accuracy on benchmark datasets such as CICDDoS2019 and CIC-IDS2017, but their reliance on labeled attack data limits their transferability: an attack family that was not represented in training is essentially invisible to the detector at inference time. Additionally, these studies typically assume access to uniquely- identifiable source addresses, an assumption that NAT and carrier-grade NAT invalidate in realistic deployment scenarios.

The second group focuses on *deep-learning* architectures for intrusion detection more broadly. Recent contributions include the transformer-based framework of Li et al. [7], the multi-class IoT-oriented system of Wahab et al. [10], and the deep reinforcement-learning approach of Satpathy et al. [9]. While these architectures deliver incremental accuracy gains, they impose substantial engineering costs: training requires GPU clusters, inference latency may exceed real-time budgets on modest hardware, and the resulting models are notoriously opaque. For a production SOC operator, the inability to explain a verdict is itself an operational liability.

The third group — closest in spirit to the present work — employs *unsupervised* or *one-class* methods that model only normal behavior. This family includes the software-defined-network detectors of Bahashwan et al. [5], the IoT-focused detection of Almaraz-Rivera et al. [3], and the hybrid blockchain-enabled firewall of Leka et al. [6]. The appeal of the one-class formulation is precisely that it sidesteps the need for a labeled attack corpus, which in turn enables deployment in environments where attack ground truth is unavailable or uncharacterized.

The present work positions itself within this third group. It contrasts with prior one-class studies in three specific ways. First, it commits to a *NAT-tolerant* feature vector from the outset, deliberately excluding source-IP-derived features that lose meaning under address translation. Second, it uses *classical* classifiers (OC-SVM and k-NN) rather than deep-learning alternatives, on the premise that interpretability and training frugality outweigh the marginal accuracy gain of heavier architectures. Third, it provides an end-to-end GPU-accelerated implementation exposed through an idiomatic Python API, bridging the gap between algorithmic research and deployable engineering.

The survey of Loaiza et al. [8] provides useful broader context on how artificial intelligence and machine learning are reshaping cybersecurity practice as a whole, situating

the DDoS-specific work above within the larger movement toward behavioral, data-driven security controls.

1.2 Background and Context

Historically, DDoS mitigation has relied on perimeter-based defenses such as access-control lists, static rate limiting, and signature-based intrusion-detection systems. These approaches depend on known attack fingerprints or on the availability of stable source identifiers (IP addresses, autonomous system numbers) that can be blacklisted when traffic anomalies are observed. Two structural trends have gradually eroded the effectiveness of such defenses. The first is the widespread deployment of Network Address Translation, carrier-grade NAT, and cloud front-ends, which collapse thousands of distinct clients behind a shared public address. Under NAT, the source IP field of a packet no longer identifies an individual host; blacklisting a single address can therefore deny service to legitimate users, while attackers can rotate through many translated addresses without ever reusing the same external identifier. The second trend is the maturation of attack tooling. Modern DDoS traffic is generated by large distributed botnets composed of compromised IoT devices, virtualized hosts, and amplification-capable open resolvers. Each individual source may contribute only a modest amount of traffic — enough to remain below simple rate thresholds — yet the aggregate load is sufficient to exhaust server capacity.

Given these trends, the research community has converged on the view that effective DDoS detection must be *behavioral* rather than *identity-based*. Instead of attempting to trace each packet to a unique source, a behavioral detector observes statistical properties of the aggregate traffic stream — packet rates, inter-arrival times, flag distributions, and connection-lifecycle completeness — and flags deviations from an empirically learned profile of normal activity. This shift aligns naturally with the machine-learning paradigm, where models are trained to distinguish two regions of a high-dimensional feature space: the region occupied by benign traffic, and the region (or rather, the complement) occupied by attacks.

1.3 Problem Statement

The specific problem addressed in this thesis can be stated as follows. Given a continuous stream of TCP packets captured at an ingress point in front of a protected server, with all source addresses potentially obscured by NAT, produce a real-time classification indicating whether the aggregate traffic currently constitutes a DDoS attack against that

server. The classifier must operate under three practical constraints: (i) it must not rely on source-IP uniqueness, since NAT invalidates this assumption; (ii) it must produce decisions at sub-minute latency so that mitigation can be triggered before the attack consumes significant resources; and (iii) it must be trainable from only a modest sample of benign traffic, because fully labeled attack corpora representative of a production network are rarely available.

1.4 Objectives

The project pursues four concrete objectives:

1. **Architect** a modular, microservice-based detection pipeline separating packet capture, feature extraction, and classification, so that each stage can be scaled or replaced independently.
2. **Implement** the two classifiers most commonly advocated for NAT-tolerant behavioral detection — k-Nearest Neighbors and One-Class Support Vector Machines — with GPU-accelerated numerical backends in C++ and CUDA.
3. **Validate** the pipeline in a reproducible GNS3 environment using representative attack families (SYN flood, HTTP flood) generated against a live target.
4. **Evaluate** detection quality with threshold-independent (ROC–AUC) and threshold-dependent (recall) metrics, and analyze sensitivity to training-data contamination.

1.5 Contributions

The contributions of the work reported in this document are threefold. First, it demonstrates that a purely behavioral, NAT-tolerant detector can be built from classical anomaly-detection algorithms without resorting to deep neural networks, provided that features are chosen carefully and scaling is handled rigorously. Second, it provides a reusable C++/CUDA algorithm library exposed to Python via `pybind11`, which decouples algorithmic performance from the orchestration layer and allows the same backend to be used from any Python-hosted analysis service. Third, it presents an empirical study of how the OC-SVM family behaves when training data is contaminated with attack traffic — a condition that is exceedingly difficult to rule out in operational networks — and shows that contamination produces silently-degenerate models that retain superficially-attractive recall while having zero or near-random discriminative power.

1.6 Thesis Structure

The remainder of this document is organized as follows. Chapter 2 specifies the functional requirements of the detection pipeline and the operational constraints under which it must run. Chapter 3 presents the architectural design, class hierarchy, and the mathematical foundations of the implemented algorithms. Chapter 4 describes the technology stack, the rationale for the chosen languages and libraries, and installation procedures. Chapter 5 documents the testing environment, the simulated attack families, the unit-test coverage strategy, and the evaluation metrics. Chapter 6 reports the experimental results, interprets them, and discusses directions for future work.

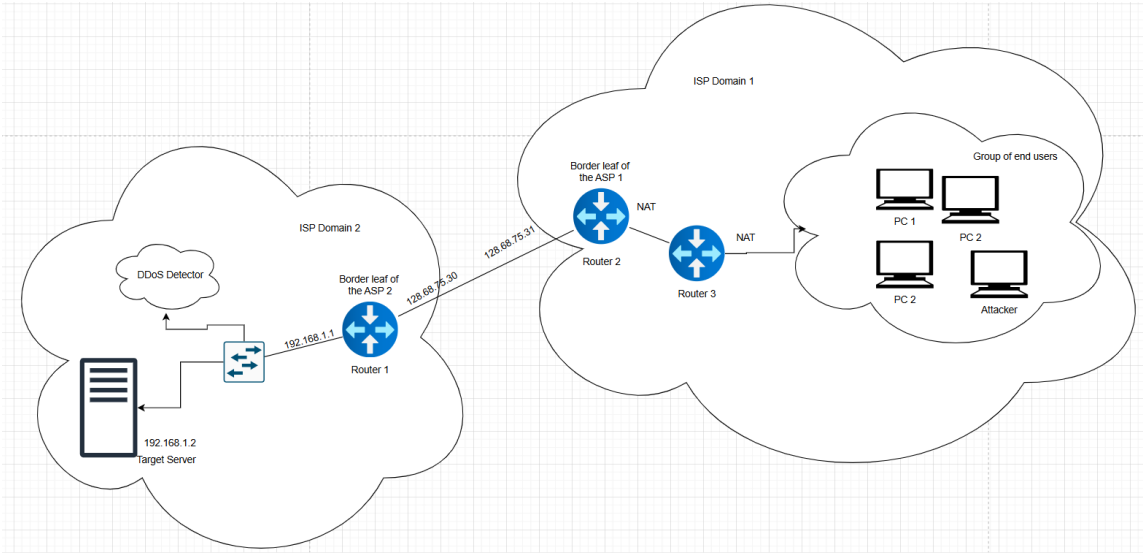

Figure 1.1: Network architecture overview.

Figure 1.1 illustrates how two Internet Service Providers (ISPs) communicate with each other. We can see a cluster of users in the domain of ISP 1; they are behind the Network Address Translation (NAT) interface of Router 3. This means that all the requests we capture in the ISP 2 domain will have a source IP of `128.68.75.31`, no matter from which user the request is coming. The proposed network setup follows a server with top-of-rack switch (ToR) architecture, commonly deployed in enterprise data centers. The target server (IP: `192.168.1.2`) is directly connected to the ToR switch. This approach enables us to connect our DDoS classifier instance directly to the switch and, by tunneling traffic to its interface, capture all requests. Tunneling is a setting on the switch that enables a certain MAC address to listen to all the traffic on the switch, regardless of whether it is the destination MAC or not.

Chapter 2

Specification of the Software Requirements and their Analysis

Because the requests are coming from behind a NAT interface, we will not be able to distinguish different IP addresses, which means we will have to gather Layer 2 data. We will need to listen on ports `80` and `443` via TCP in order to obtain traffic data.

2.1 Requirements and their Analysis

2.1.1 Start/Stop Capture

The Start/Stop Capture functionality represents the first operational feature of the DDoS detection system. It allows the system administrator or security engineer to control packet capture operations through RESTful endpoints exposed by the FastAPI-based Network Sniffer microservice. The endpoint `/open_socket` initiates the packet capture process by specifying the network interface to listen on, capturing only TCP packets targeting ports `80` (HTTP) and `443` (HTTPS).

The packet capture process is performed at the kernel level to collect real-time network traffic data. Each packet is timestamped and forwarded to the feature computation module for further aggregation. The `/close_socket` endpoint stops the capture process gracefully, ensuring that any packets stored in the buffer of the machine are flushed and that system resources such as sockets and file descriptors are properly released. This mechanism provides flexible management of data acquisition while preventing resource exhaustion and ensuring accurate data gathering.

2.1.2 Feature Computation

Feature computation is at the core of the data processing pipeline. The captured packets are aggregated into 5-second time intervals. For each active IP address, a set of numerical and statistical features is extracted to represent the behavioral characteristics of the traffic during that time frame. Typical features include packet rate, byte rate, and the ratio of TCP flags (SYN, ACK). The system maintains a rolling window, allowing temporal context for the machine learning models to evaluate ongoing trends. This design balances responsiveness and stability — short enough to detect attacks quickly, but long enough to minimize false positives from short spikes in traffic.

Feature computation runs as a background task, automatically triggered at each bucket interval. It ensures synchronization between the capture timestamp and the analyzer's feature timeline. Any missing packets or dropped intervals are logged for diagnostic purposes.

Feature Catalog

The concrete feature vector computed for each IP–window pair comprises the following quantities:

- **Packet rate** (f_1): the number of TCP packets observed during the five-second window, divided by the window length. Under benign conditions this rate reflects the user's browsing tempo; under a flood it is sharply elevated.
- **Byte rate** (f_2): the cumulative byte count over the window, again normalized by window length. Byte-rate spikes without a corresponding jump in packet rate can indicate amplification attacks, while the opposite pattern suggests small-packet floods typical of SYN storms.
- **SYN count** (f_3) and **SYN ratio** (f_4): the absolute number of packets with the SYN flag set, and the fraction of all packets in the window carrying SYN. A healthy TCP session contains exactly one SYN; a SYN flood will push f_4 close to unity.
- **ACK count** (f_5) and **ACK ratio** (f_6): analogous statistics for the ACK flag. The ratio $f_3/(f_5+\varepsilon)$ captures the handshake-completion imbalance that is characteristic of half-open-connection attacks.
- **Unique source-port count** (f_7): the number of distinct ephemeral source ports observed. Because NAT aggregates many clients under a single IP, the diversity of ephemeral ports becomes the most reliable surrogate for client-count; a sudden collapse of this diversity often signals a single-source bot.

- **Mean inter-arrival time** (f_8) and **inter-arrival variance** (f_9): captured as first-order temporal statistics. Attack traffic tends to exhibit very low mean inter-arrival times coupled with low variance, whereas human browsing produces bursty, high-variance arrivals.

These nine features form the basis of every feature vector $x \in \mathbb{R}^9$ consumed by the downstream classifiers. The choice to keep the feature dimensionality modest is deliberate: it keeps distance-based algorithms (such as k-NN) tractable, preserves interpretability for the network engineer reviewing alerts, and reduces the exposure of the system to the curse of dimensionality.

2.1.3 Feature Delivery API

The Feature Delivery API is the interface between the data collection microservice and external analytical tools. The endpoint `/get_data/{ip}` enables the retrieval of the latest N feature vectors associated with a particular destination IP address. This API serves as the bridge between the raw packet data and the machine learning inference engine.

It ensures that the data returned is normalized and timestamp-aligned. The response is formatted in JSON, containing each feature name and value, time range, and version metadata for traceability. Engineers and downstream systems can use this endpoint to visualize traffic behavior trends or feed external monitoring dashboards.

2.1.4 Analyze on Demand

The Analyze on Demand functionality allows an engineer or automated monitoring system to query the analyzer microservice for an immediate DDoS status evaluation. By calling `/estimate_attack?ip=...`, the analyzer fetches the latest set of feature vectors for the target IP and passes them to the C++ classification library through the `pybind11` interface.

The library executes the One-Class Support Vector Machine (OC-SVM) and k-Nearest Neighbors (KNN) models, returning a predicted label. These models have been trained using previous attack-labeled data; the output includes the decision label (`UNDER_ATTACK` or `CLEAR`).

The service compiles these results into a structured JSON response including model version and timestamp, enabling traceability and auditability of the decision-making process.

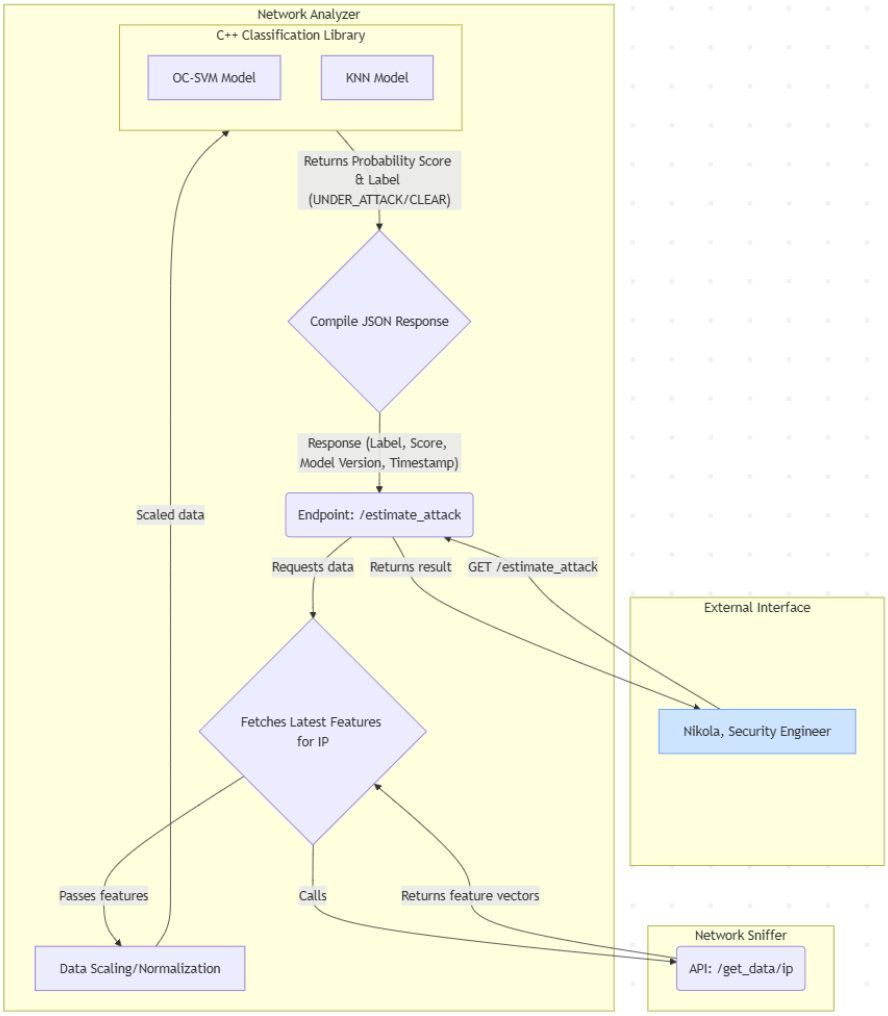

Figure 2.1: Feature Delivery Process.

2.1.5 Continuous Monitoring

Continuous monitoring ensures that DDoS detection occurs automatically and in real time. Rather than requiring manual requests, the analyzer microservice executes periodic classification cycles synchronized with the feature computation intervals (every 5 seconds). Each monitored IP is analyzed automatically as new feature data becomes available.

The system maintains a state for each IP, storing its current classification (e.g., `CLEAR`, `UNDER_ATTACK`). When a state change is detected, an alert or a different connected system can be triggered. This allows for immediate response and correlation across network management systems. Continuous monitoring transforms the DDoS detector into an autonomous early-warning system, capable of detecting emerging attacks with minimal human intervention.

2.1.6 Get Current State

This functionality provides an on-demand summary of the current security status of any monitored IP. The endpoint `/ongoing_attack/{ip}` returns a structured response containing the latest classification label, the detection confidence score, and the timestamp of the last analysis.

This feature is particularly useful for integrating with external network monitoring dashboards, automated mitigation systems, or SOC tools. It provides a clear and machine-readable status snapshot, enabling operators to quickly verify whether an IP is currently under attack or behaving normally.

2.1.7 Model Management

The Model Management functionality guarantees that the machine learning models used for DDoS detection (OC-SVM and KNN) are always current and consistent. Upon startup, the analyzer microservice loads both models from disk through the C++ library. Such an option helps to load pre-trained models at the start-up time of the service and to update the model's settings without the need to restart. This dynamic model management enables rapid iteration and retraining without redeploying the entire system — an essential capability in evolving threat environments.

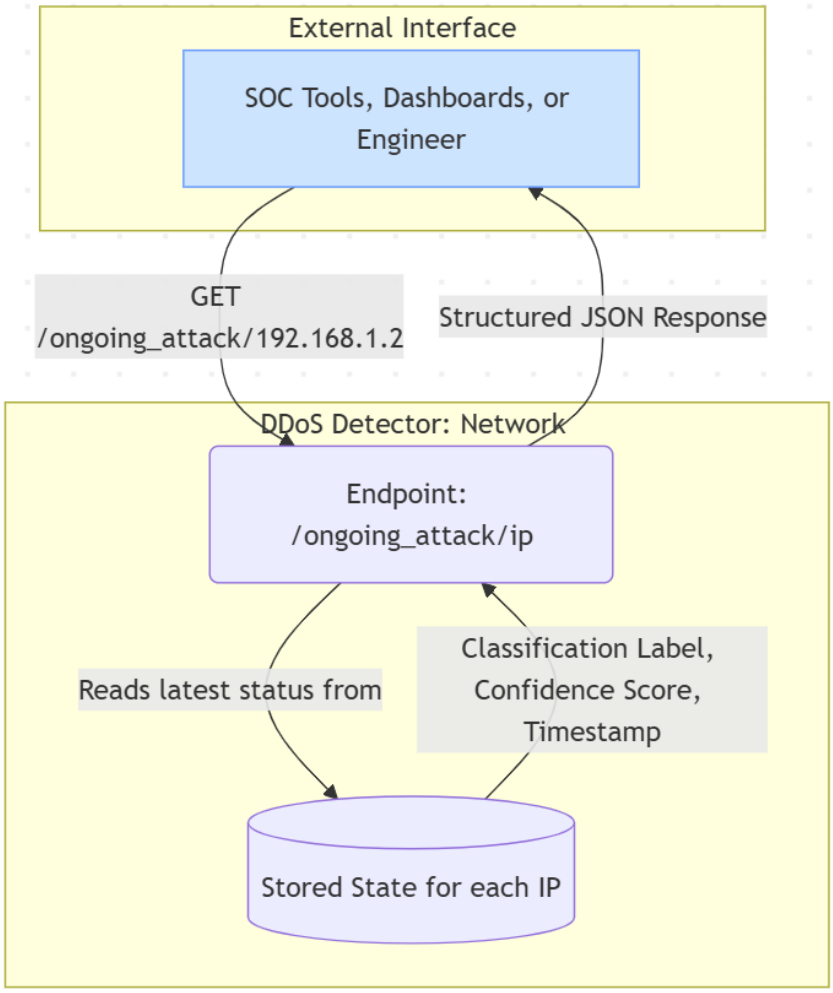

Figure 2.2: Get Current state Overview.

2.1.8 Data Scaling

Before feeding network traffic features into the OC-SVM and KNN models, it is essential that all input variables are transformed into a normalized scale to prevent feature-magnitude bias. This requirement specifies that each computed feature — such as packet rate, byte rate, and SYN/ACK ratios — must undergo standardization using parameters (mean and standard deviation) derived from the training dataset.

Formally, for each feature x_i the standardized value $\tilde{x}_i$ is computed as:

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}, \tag{2.1}$$

where μ_i and σ_i denote the mean and standard deviation of feature i in the training dataset, respectively. During real-time analysis, the system applies the same scaling coefficients to ensure consistency between training and inference distributions.

The scaling ensures that no individual metric dominates the model's decision due to numerical range differences. These normalization parameters are stored in a configuration file (e.g., `config.txt`) within the Network Analyzer microservice and are loaded automatically by the analyzer at startup.

Why Scaling Matters: A Worked Example

Consider two features drawn from a typical traffic window: packet rate f_1, which may range from roughly 10 to 10,000 packets per second, and SYN ratio f_4, which is bounded between 0 and 1. In an unscaled Euclidean distance computation, the packet-rate dimension contributes a squared term on the order of 10^8, whereas the SYN ratio contributes at most 1. Any k-NN comparison or RBF kernel evaluation will therefore be dominated almost entirely by differences in packet rate, and the SYN-ratio information — which is often the more discriminative signal — will be effectively discarded. After standardization using Equation 2.1, both features lie on a shared scale with approximately zero mean and unit variance, so each contributes proportionally to the distance metric.

Train–Test Parameter Consistency

It is essential that the scaling parameters μ_i and σ_i used at inference time are *exactly* those computed on the training set. Recomputing these statistics from the live traffic stream would produce a moving reference distribution, and any sufficiently large attack would end up re-centering the scaler around the attack itself — after which the attack would appear "normal" in the scaled coordinates. This subtle failure mode motivates the

choice to persist (μ_i, σ_i) on disk as part of the model artifact and to refuse inference if the configuration file is missing or its checksum does not match the expected value.

2.2 Constraints

2.2.1 Microservice Containerization

Due to compatibility limitations between VMware Workstation and containerization technologies such as Docker, WSL2, and Hyper-V, the system must be built and executed natively on Windows 10 without relying on any form of containerization. Docker and similar virtualization layers must not be used or required for deployment, as VMware Workstation is essential for running the GNS3 network simulator that is used in the testing and validation environment.

Chapter 3

Design of the Software Solution

3.1 Project Architecture Overview

The design of the DDoS detection application is shaped by several key contextual and operational constraints. Since all traffic originates from behind a NAT, it is impossible to distinguish unique public IP sources. Therefore, the system captures and analyzes packet-level attributes such as TCP flags, bytes per second, and connection diversity metrics (e.g., SYN count). The entire system is implemented to run natively on Windows 10 without the use of Docker, WSL2, or other containerization tools, since compatibility with VMware Workstation is required for integration with the GNS3 network simulator.

Services such as packet capture, feature computation, and attack analysis are deployed as independent microservices. Network traffic is captured through the Npcap library for low-level packet access. To support near-real-time detection, all components process data in 5-second intervals. The separation of data capture from analysis ensures that packet acquisition continues even if the analytical subsystem restarts or fails, in which case a persistent store can be added.

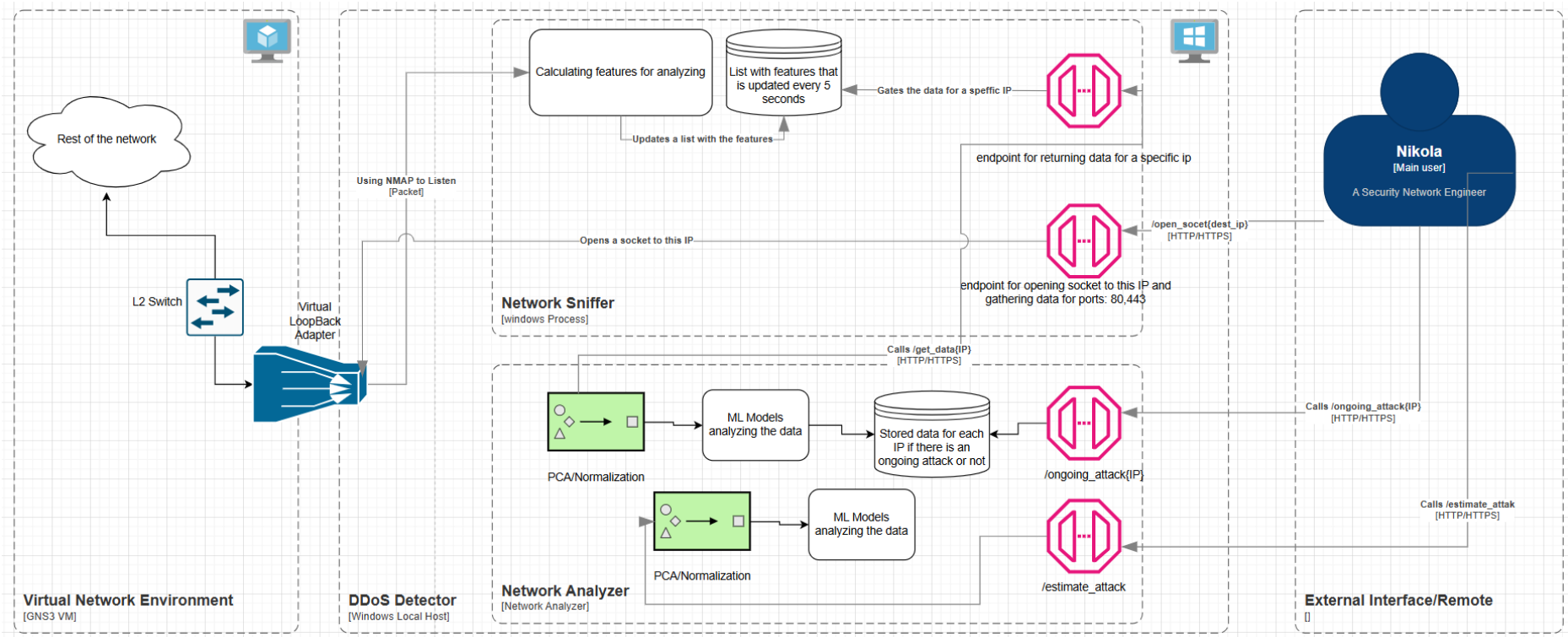

Figure 3.1: Micro services Architecture Overview.

The microservice architecture shown in Figure 3.1 is designed to meet the functional requirements of a DDoS detection system by integrating modular components that handle data collection, processing, and analysis in real time. The environment is divided into three main layers: the Virtual Network Environment used for testing a real DDoS attack, the Network Sniffer microservice, and the DDoS Analyzer microservice, all orchestrated through RESTful APIs.

In the Virtual Network Environment, traffic generated from the simulated GNS3 setup is directed toward a target server (`192.168.1.2`). This setup replicates a real-world Internet topology where an attacker and legitimate users reside in different ISP domains. A SPAN or mirrored port forwards copies of network packets to the Network Sniffer microservice for inspection. This forms the entry point for traffic monitoring and DDoS detection.

The Network Sniffer microservice serves as the system's data acquisition and pre-processing layer. Built using FastAPI, it provides two critical operational endpoints — `/open_socket` and `/close_socket` — that allow administrators to start and stop packet capture dynamically. The capture process operates at the kernel level, filtering only TCP packets destined for ports `80` and `443` (HTTP and HTTPS), ensuring efficient monitoring of web traffic. When started, packets are timestamped and sent to the feature computation module; when stopped, any buffered data is flushed to prevent resource leakage.

Feature computation forms the analytical core of the Network Sniffer service. Captured packets are aggregated into five-second intervals, during which numerical and statistical features such as packet rate, byte rate, and TCP flag ratios (SYN/ACK) are calculated per active IP address. This rolling window allows the system to retain temporal context — short enough to respond quickly to attacks, yet stable enough to avoid false positives caused by transient spikes or momentary packet jitter. Synchronization between capture timestamps and feature intervals ensures data accuracy.

The Feature Delivery API bridges the Network Sniffer and external systems such as the DDoS Analyzer. Through the `/get_data/{ip}` endpoint, the API provides normalized, timestamp-aligned feature vectors in JSON format, complete with feature names, time ranges, and version metadata. This enables interoperability with visualization dashboards, monitoring tools, and machine learning modules that rely on consistent data for analysis and decision-making.

The DDoS Analyzer microservice is responsible for detecting and classifying potential attacks. It can perform both on-demand and continuous analysis. When the `/estimate_attack?ip=...` endpoint is called, the analyzer fetches recent features for the target IP, scales them using stored normalization parameters, and forwards them to a C++ classification library via a `pybind11` interface. The library executes OC-SVM and

KNN algorithms, which have been trained on labeled historical attack data. The output includes a classification label (either `UNDER_ATTACK` or `CLEAR`), along with timestamps and model version identifiers for auditability.

For automated protection, the analyzer includes a continuous monitoring mode, running every five seconds in sync with feature generation. Each monitored IP (target host) is automatically analyzed as new data becomes available, maintaining a continuous state that tracks whether an address is currently under attack or behaving normally. When a state change occurs, alerts or external triggers can be sent to integrated systems such as security-operation centers or mitigation services, enabling rapid incident response.

The Get Current State endpoint (`/ongoing_attack/{ip}`) provides a quick snapshot of any monitored IP's latest classification, including the detection confidence score and timestamp of the last analysis. This supports easy integration with SOC dashboards or automated mitigation platforms, offering clear visibility into the network's current security posture.

Model management is another vital aspect of the system. Upon startup, the analyzer loads its machine learning models (OC-SVM and KNN) directly from disk through the C++ library. This allows for dynamic model updates without restarting the service, enabling frequent retraining or tuning as new attack data becomes available. Alongside the models, a configuration file (e.g., `config.txt`) stores the normalization coefficients used for feature scaling, ensuring consistent behavior between training and real-time inference.

Finally, the deployment constraint requires the system to run natively on Windows 10 without containerization. This design choice arises from compatibility issues between VMware Workstation — used to host the GNS3 simulation — and container technologies such as Docker, WSL2, or Hyper-V. As a result, all components, including FastAPI services and the C++ inference engine, are deployed directly on Windows. The sniffer interacts with the network interface through native Windows drivers, while the analyzer runs its Python and C++ processes within the same environment.

Overall, the microservice architecture effectively meets the DDoS detection system's functional requirements through a combination of modular decomposition, scalability, and real-time analytics. The Network Sniffer ensures accurate and efficient packet collection and feature extraction; the Analyzer provides both manual and automated attack detection using advanced models; and the REST APIs offer interoperability and traceability. Together, they form a reliable, self-contained detection framework that operates seamlessly in a virtualized Windows-based testing environment.

3.2 Class Architecture

The system is organized around a hierarchical structure that separates general algorithmic responsibilities from the specific logic required by individual learning methods. At the foundation of this structure is an abstract component that defines the essential capabilities expected of any model within the framework. It maintains information about the input features, the target variable, and the size of the training data. It also manages memory on the GPU for storing training inputs and targets and provides optional preprocessing mechanisms such as feature scaling and dimensionality reduction. This foundational component establishes a uniform interface for training and prediction that all specialized models must follow, ensuring consistency across the framework.

Two concrete learning components extend this foundation, each implementing a distinct method while relying on the shared services provided at the base. One represents a nearest-neighbor approach. It stores a parameter that determines how many neighbors are considered when making a prediction and uses the common data-preparation mechanisms inherited from the base. Its training phase relies primarily on setting up the data properly, while its prediction phase involves identifying the closest stored examples to each new sample. The actual computations involved in this process are delegated to a dedicated helper that runs on the GPU, enabling efficient distance calculation and neighbor selection.

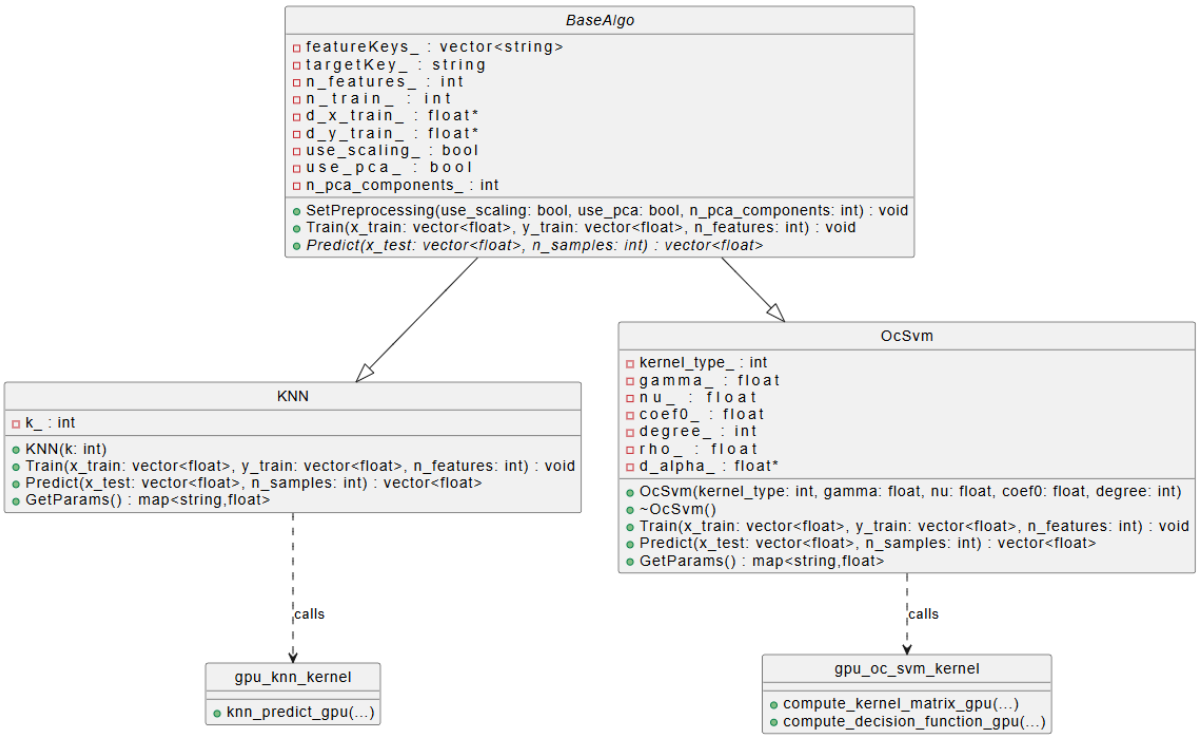

Figure 3.2: UML Class Diagram.

The second specialized component implements a one-class classification method aimed at distinguishing normal samples from anomalous ones. It introduces a set of additional parameters that define the behavior of the underlying decision function, such as kernel type, kernel coefficients, and learned values associated with the resulting boundary. Like the nearest-neighbor component, it uses the shared capabilities provided by the foundational layer to handle input data and apply any configured preprocessing. During training, it constructs the decision boundary using a helper that performs kernel computation and optimization on the GPU. During prediction, it again relies on this helper to evaluate new samples against the learned boundary.

Supporting these two learning methods are two GPU-based helper modules. Although they are represented independently, they do not participate in the class hierarchy. Instead, they serve as computational engines that the learning components invoke during training and prediction. One provides procedures for computing distances and neighbor relationships for the nearest-neighbor method, while the other computes kernel matrices and decision-function values required by the one-class classifier. Their presence allows the main learning components to offload intensive numerical operations to the GPU while maintaining a clear separation between high-level algorithmic structure and low-level computational routines.

3.3 Implemented Algorithms

Anomaly detection plays a central role in modern cybersecurity, especially when it comes to identifying Distributed Denial-of-Service (DDoS) attacks. These attacks work by overwhelming a target — typically a server or network — with an enormous volume of traffic originating from many different sources. What makes DDoS attacks particularly challenging is that they often begin subtly, blending in with regular network activity before ramping up to damaging levels. Because of this, the ability to detect even small deviations from normal network behavior can make the difference between early mitigation and a full-blown service outage.

To tackle this challenge, machine learning algorithms that model normal network behavior — without relying on large datasets of labeled attacks — are extremely valuable. This is where k-Nearest Neighbors (k-NN) and One-Class Support Vector Machines (OC-SVM) come into play. Both are well suited for anomaly detection, but they approach the problem from fundamentally different angles, making them excellent candidates for benchmarking against each other.

The k-NN algorithm offers a very intuitive approach based on local density. It looks at how close a given network observation is to its nearest neighbors in historical traffic.

Under normal conditions, traffic tends to form clusters: similar packet rates, flow sizes, connection durations, and so on. When a point appears far from these clusters — meaning its k closest neighbors are unusually distant — it is flagged as suspicious. This makes k-NN especially effective at detecting irregular traffic spikes or unusual connection patterns that may hint at a DDoS attack. Importantly, k-NN does not assume anything about the data distribution, which is ideal in networks where traffic patterns can vary widely and unpredictably.

In contrast, the OC-SVM takes a more global and mathematically formal approach. Instead of focusing on local relationships, it tries to learn the overall shape of what "normal" traffic looks like. By using kernel functions, it can model complex patterns and create a boundary that tightly encloses the normal data. Anything falling outside this learned boundary is considered an anomaly. This makes OC-SVM particularly powerful when labeled data is limited — a common situation in cybersecurity — because it can generalize well from only normal traffic samples. It also tends to be more robust to small fluctuations and noise, which helps reduce false alarms in busy or volatile network environments.

Benchmarking k-NN and OC-SVM side by side provides valuable insight into the trade-offs between local sensitivity and global generalization. k-NN is better at spotting sudden, localized deviations, such as an abrupt spike in packet size or a surge of connections from a single host — typical signs of early-stage DDoS activity. OC-SVM, on the other hand, excels at understanding the broader structure of network traffic, making it more resilient to noise and better at detecting slow-burning or distributed attacks that subtly shift overall traffic characteristics.

By comparing these two methods, we gain a deeper understanding of how different modeling assumptions impact detection performance under various network conditions. This ensures a more complete assessment of anomaly detection strategies and helps identify which approach — or combination of approaches — is best suited for real-world DDoS detection.

3.3.1 One-Class Support Vector Machines (OC-SVM)

The One-Class Support Vector Machine (OC-SVM) is a powerful method for anomaly detection, especially in situations where only examples of what "normal" looks like are available. Instead of relying on labeled anomalies — which are often rare, expensive to obtain, or simply unknown — the OC-SVM tries to learn the shape of the normal data distribution and then determines whether new observations fit inside that shape or fall outside of it.

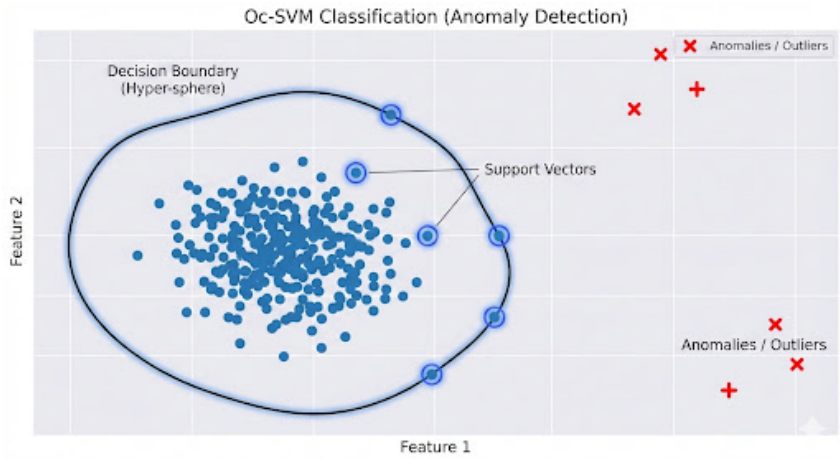

Figure 3.3: Oc-SVM Classification.

The OC-SVM uses a kernel function to map the data into a high-dimensional feature space, where it attempts to separate the normal data from the origin with as simple a boundary as possible. In this transformed space, the algorithm tries to draw a surface that encloses most of the training points. When the algorithm is done, it has essentially learned a function that outputs positive scores for points that fall inside the learned region (i.e., points that look normal) and negative scores for points that lie outside (potential anomalies).

Formally, given a training set $\{x_1, x_2, \ldots, x_n\} \subset \mathbb{R}^d$, the OC-SVM solves the following quadratic optimization problem:

$$\begin{aligned} \min_{w,\xi,\rho} \quad & \frac{1}{2}\|w\|^2 + \frac{1}{\nu n}\sum_{i=1}^{n}\xi_i - \rho \\ \text{subject to} \quad & \langle w, \phi(x_i)\rangle \geq \rho - \xi_i, \quad \xi_i \geq 0, \end{aligned} \tag{3.1}$$

where $\phi(\cdot)$ is the feature map induced by the chosen kernel and $\nu \in (0, 1]$ is a hyperparameter that controls the trade-off between boundary tightness and the expected fraction of outliers.

The decision function then takes the form:

$$f(x) \;=\; \operatorname{sgn}\left(\sum_{i=1}^{n}\alpha_i\, K(x_i, x) - \rho\right), \tag{3.2}$$

where $K(x_i, x)$ is the kernel function and α_i are the dual coefficients learned during training. A smaller ν means the model assumes very few anomalies exist in the training data and tries to tightly wrap the boundary around them. A larger ν allows more points to lie outside the boundary, making the model more tolerant of noise.

Dual Formulation

The primal problem in Equation 3.1 is converted to its Lagrangian dual in order to expose the kernel function and admit non-linear decision boundaries. The dual is:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j\, K(x_i, x_j) \\ \text{subject to} \quad & 0 \le \alpha_i \le \frac{1}{\nu n}, \quad \sum_{i=1}^{n}\alpha_i = 1. \end{aligned} \tag{3.3}$$

After solving this quadratic program, the offset ρ is recovered from any training point x_k whose dual coefficient lies strictly inside its box constraint ($0 < \alpha_k < 1/(\nu n)$) via $\rho = \sum_i \alpha_i K(x_i, x_k)$.

Interpreting the ν-Property

One of the most useful properties of the OC-SVM is that the parameter ν simultaneously bounds two quantities: it is an *upper bound* on the fraction of training examples allowed to lie outside the learned boundary ("outliers"), and a *lower bound* on the fraction of training examples whose dual coefficient is non-zero ("support vectors"). This dual interpretation gives the network engineer an intuitive knob: if prior knowledge suggests that the benign baseline contains about 2% noisy samples, ν can be set to 0.02 and the model will respect this expectation by construction. In the experiments reported in Chapter 6, ν was fixed at 0.05 as a mild over-estimate of the expected contamination rate.

3.3.2 Kernel Functions Used for OC-SVM

Linear Kernel

The linear kernel is the most straightforward kernel function used in a One-Class SVM. It is simply the dot product of two vectors, meaning it measures similarity based on how aligned two points are in the original feature space:

$$K_{\text{lin}}(x, y) \;=\; \langle x, y\rangle \;=\; x^{\top} y. \tag{3.4}$$

Because no nonlinear transformation is applied, the OC-SVM with a linear kernel attempts to separate the normal data from the origin using a single flat hyperplane.

The linear kernel tends to work best when the distribution of normal data is approximately convex and does not require curved or complex boundaries to enclose it. If the

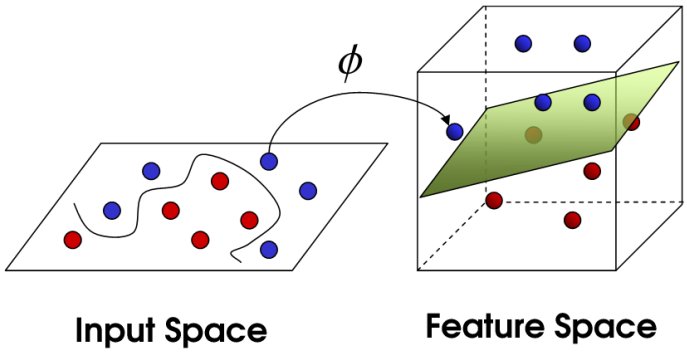

Figure 3.4: Linear Kernel Function.

normal region can be reasonably captured by a flat separating surface, then a linear kernel is not only adequate but often ideal. This is especially true in high-dimensional, sparse datasets, such as text representations (TF–IDF vectors, bag-of-words models) or certain types of document and log data.

However, the linear kernel's simplicity is also its most significant limitation. It has no ability to model curved or irregular shapes in the data distribution. If the boundary that separates normal from abnormal data is inherently nonlinear — such as data forming clusters, rings, or spirals — the linear kernel can easily underfit. In such cases, more expressive kernels like the RBF or polynomial kernel are usually preferred.

Polynomial Kernel

The polynomial kernel adds a layer of expressive power by taking into account not only the original features but also their interactions:

$$K_{\text{poly}}(x, y) \;=\; \big(\gamma \langle x, y\rangle + c_0\big)^d, \tag{3.5}$$

where γ is a scaling coefficient, c_0 is an independent term, and d is the polynomial degree. This expansion allows the OC-SVM to draw curved, flexible boundaries around the normal data.

In the context of OC-SVM, the polynomial kernel is especially helpful when the normal behavior in the dataset is shaped by feature interactions. Quadratic or cubic boundaries, which arise from second- or third-degree polynomials, can outline regions of the data that are curved, elliptical, or shaped by complex dependencies between features.

However, with this flexibility comes a significant trade-off. As the degree of the polynomial grows, the decision boundary becomes increasingly complex. A high-degree polynomial can fit extremely detailed shapes in the data, but it can also become overly

responsive to small fluctuations or noise, causing overfitting. Choosing a degree that is too low might not provide enough capacity to capture the true structure of the normal region, leading to underfitting.

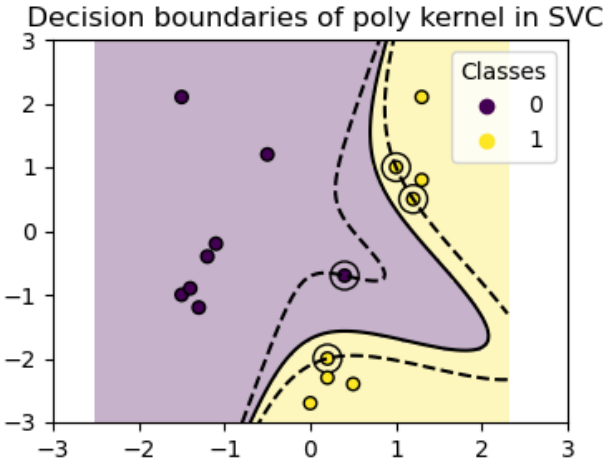

Figure 3.5: Poly Kernel.

Finding the right degree for the polynomial kernel therefore requires careful consideration. It depends on the underlying nature of the dataset: if normal behavior results from meaningful interactions among features, a moderate-degree polynomial can be powerful and effective. But if the feature relationships are simple or the data is noisy, a polynomial kernel can quickly become too rigid or too sensitive.

Radial Basis Function (RBF) Kernel

The Radial Basis Function (RBF) kernel is one of the most powerful and commonly used kernels in OC-SVM because it focuses on local similarity:

$$K_{\mathrm{RBF}}(x, y) \;=\; \exp\!\left(-\gamma \left\|x - y\right\|^2\right), \quad \gamma > 0. \tag{3.6}$$

Points near each other are considered highly similar, while similarity drops off quickly with distance. This allows the OC-SVM to form smooth, curved boundaries that naturally wrap around the true shape of the data.

Conceptually, the RBF kernel lifts the data into an infinite-dimensional feature space, but the kernel trick makes this practical by computing similarities directly without constructing that space. This ability to model nonlinear patterns is especially valuable in anomaly detection, where normal behavior rarely follows simple geometric shapes.

A key parameter of the RBF kernel is γ. A small γ value leads to broad, smooth boundaries that risk overlooking fine details, while a large γ makes the model very sensitive to small fluctuations and can cause overfitting. The main drawback of the RBF

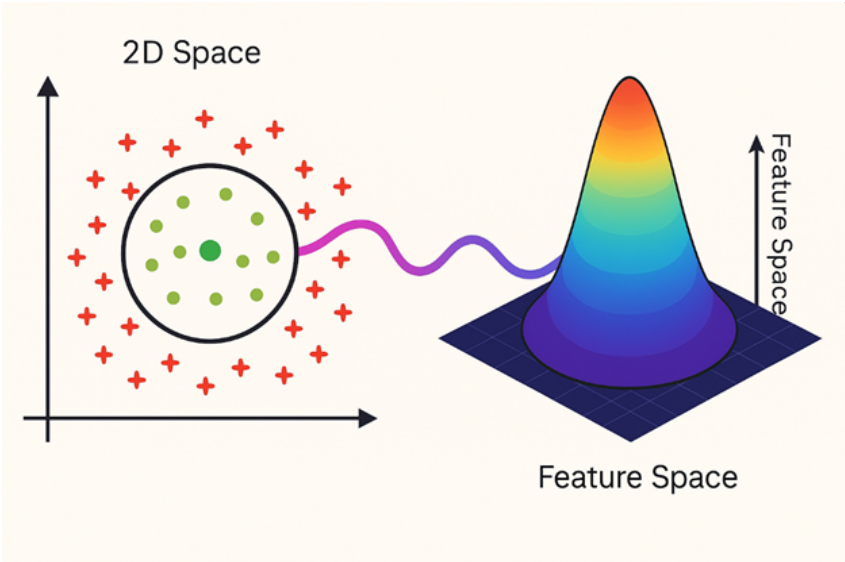

Figure 3.6: Radial Basis Function: 2D -> Hyper Space.

kernel is its computational cost, since it requires computing pairwise similarities between points. GPU acceleration — as used in this system — significantly reduces this cost.

Sigmoid Kernel

The Sigmoid kernel — often called the hyperbolic tangent kernel — is defined as:

$$K_{\text{sig}}(x, y) \;=\; \tanh\big(\gamma \langle x, y\rangle + c_0\big). \tag{3.7}$$

Unlike the RBF kernel, which measures similarity using Euclidean distance, the Sigmoid kernel models similarity through activation-like responses.

From a geometric standpoint, the Sigmoid kernel creates decision boundaries with global influence. While distance-based kernels typically form closed, localized hyperspheres, the Sigmoid kernel generates open, hyperbolic surfaces. As a result, even points located far from the dense regions of the feature space can meaningfully affect how the boundary curves. In DDoS detection, this property enables a One-Class SVM to identify anomalies by detecting changes in activation patterns.

The effectiveness of this approach in anomaly detection depends heavily on tuning two key hyperparameters: ν (nu) and γ (gamma). The ν parameter acts as a regularization control, setting an upper limit on training errors and a lower limit on the proportion of support vectors. When the model is trained on benign network traffic, ν essentially represents the expected contamination rate, so it is typically kept low (around 0.01–0.05) to keep false positives down.

The γ parameter determines the kernel coefficient and influences the complexity of the

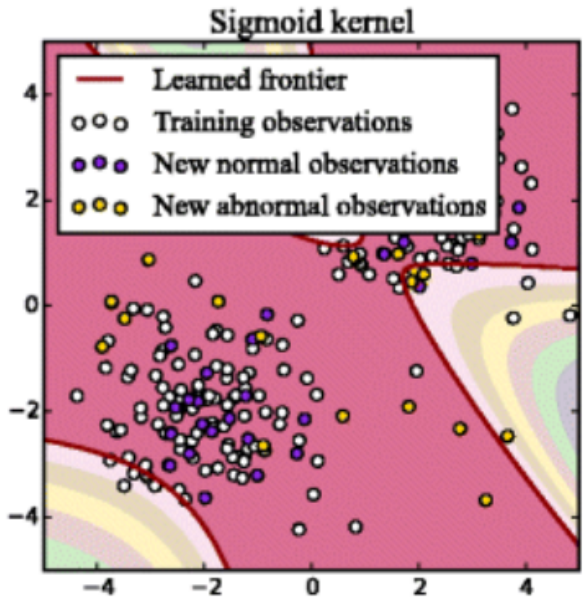

Figure 3.7: Sigmoid kernel classification.

decision boundary. If γ is set too high, the model becomes overly sensitive and overfits to noise; if too low, the decision boundary becomes nearly linear and fails to capture the non-linear signatures of more advanced attacks.

Comparative Summary of the Four Kernels

Table 3.1 consolidates the trade-offs among the four kernel families implemented in this work. The entries reflect both the published theoretical properties of each kernel and the empirical behavior observed on the GNS3 testbed (Chapter 6).

Table 3.1: Comparative properties of the four OC-SVM kernel functions.

	Linear	**Polynomial**	**RBF**	**Sigmoid**
Boundary shape	Flat	Curved	Smooth closed	Open hyperbolic
Hyperparameters	ν	ν, γ, c_0, d	ν, γ	ν, γ, c_0
Tuning difficulty	Low	High	Medium	High
Training cost	Low	Medium	High	High
Inference cost	$O(n_{sv}d)$	$O(n_{sv}d)$	$O(n_{sv}d)$	$O(n_{sv}d)$
Nonlinear capacity	None	Moderate	High	Moderate
Robustness to noise	High	Low	Medium	Low
High-dim suitability	Excellent	Poor	Good	Fair
Observed ROC–AUC	0.95	0.95	0.98	not deployed

In practice, the RBF kernel emerged as the most reliable choice for this particular feature space: its ability to wrap arbitrary closed regions around the benign cluster made it resilient to the natural variability in browser-generated traffic, while the locality of its

similarity measure prevented distant outliers from distorting the boundary. The linear kernel, despite its modeling austerity, performed nearly as well and carries the advantage of producing an interpretable weight vector that can be inspected for diagnostic purposes.

3.3.3 k-Nearest Neighbor (KNN)

The k-Nearest Neighbors (k-NN) algorithm, when applied to anomaly detection, takes a very direct and intuitive approach. Unlike model-based methods such as One-Class SVM, k-NN does not build an abstract model or learn a decision boundary. Instead, it is a brute-force, instance-based method. This means that during prediction, the algorithm simply compares each new data point to all points in the training set and bases its decision on these distance comparisons. There is practically no "training" phase; the training data is simply stored and later used as a reference.

The brute-force nature of k-NN becomes especially clear in anomaly detection. For a test point x, the algorithm computes its Euclidean distance to every training sample:

$$d(x, x_i) = \sqrt{\sum_{j=1}^{d} (x^{(j)} - x_i^{(j)})^2}, \tag{3.8}$$

identifies the k closest samples, and computes an anomaly score. A common choice is the mean distance to the k nearest neighbors:

$$s(x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} d(x, x_i), \tag{3.9}$$

where $\mathcal{N}_k(x)$ denotes the index set of the k nearest neighbors of x in the training set. The fundamental idea is that normal points live in dense regions, surrounded by many similar observations, while anomalies tend to be isolated, sitting far from their neighbors.

This makes k-NN particularly well suited for anomaly detection because it does not assume any specific data distribution or global shape. Many real-world datasets — especially those involving human behavior, network traffic, sensor data, or fraud patterns — are highly irregular and do not follow neat, Gaussian-like structures. k-NN naturally adapts to complex, nonlinear, non-convex patterns because its decisions depend purely on local density rather than on a global optimization function.

However, this simplicity and flexibility come with several downsides. First, the brute-force comparison against all training samples makes k-NN computationally expensive, especially for large datasets. Every time we classify a new point, we must compute potentially thousands or millions of distances. Without GPU acceleration or smart indexing structures, this can quickly become impractical.

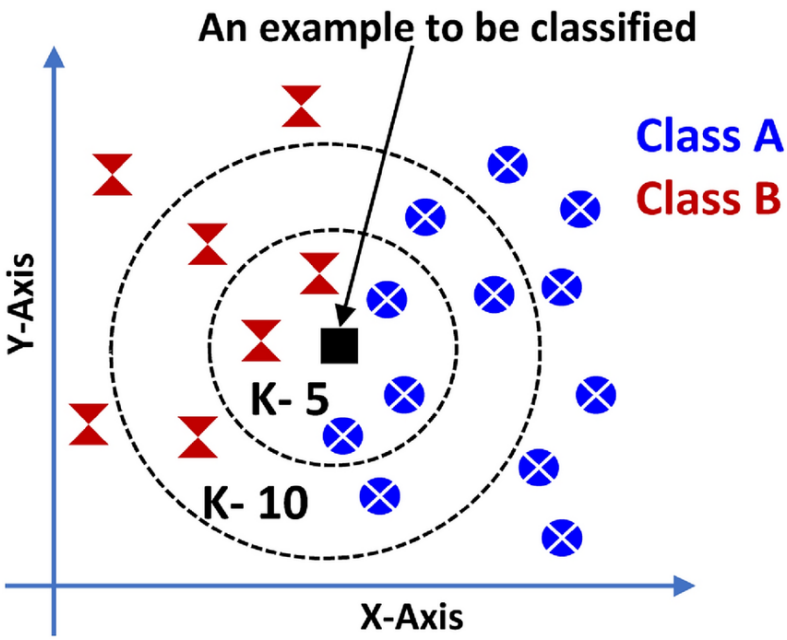

Figure 3.8: KNN Classification.

Another major challenge is that k-NN is very sensitive to feature scaling. Because its decisions rely on distance metrics, features with larger numeric ranges can dominate the outcome unless the data is properly normalized. This is why scaling and sometimes dimensionality reduction (such as PCA) are essential when using k-NN.

The choice of k also has a significant impact. A very small k may be overly sensitive to noise and produce unstable decisions, while a very large k may smooth out local structures and miss subtle anomalies. Picking the right k requires experimentation and depends on the dataset's characteristics.

Finally, because k-NN relies entirely on the stored training data, it is vulnerable to issues such as memory usage, slow prediction times, and sensitivity to irrelevant or redundant features. High-dimensional data can make distance comparisons less meaningful, a phenomenon known as the *curse of dimensionality*, which can degrade the algorithm's ability to distinguish between normal and abnormal points.

Despite these limitations, k-NN remains a widely used and intuitive method for anomaly detection, especially when combined with GPU acceleration, proper preprocessing, and thoughtful parameter tuning.

Computational Complexity

Let n denote the number of training samples, d the feature dimensionality, and m the number of test points to be classified in a batch. A naive CPU implementation performs $O(nmd)$ arithmetic operations plus $O(nm)$ comparisons to identify the top-k neighbors,

with a memory footprint of $O(nd)$ for the training set plus $O(nm)$ for the pairwise-distance matrix. In the present system, a typical operational batch consists of the N most recent feature vectors for one target IP ($m \approx 12$ for a one-minute window at five-second granularity), against a stored benign baseline of $n \approx 10{,}000$ samples and $d = 9$ features. This yields roughly 10^6 floating-point operations per batch — well within real-time budgets on the GPU, where the entire distance matrix can be computed in a single CUDA launch with thread blocks of 256 threads covering all (i, j) pairs in parallel.

Worked Example

Suppose the standardized benign baseline includes three representative training points at coordinates $(0.1, 0.2)$, $(-0.3, 0.0)$, and $(0.2, -0.1)$ in a two-dimensional projection, and that a new test vector arrives at $(2.8, 3.1)$ — far outside the typical cluster. With $k = 2$, the two nearest distances are approximately $\sqrt{(2.8-0.2)^2 + (3.1-(-0.1))^2} \approx 4.12$ and $\sqrt{(2.8-0.1)^2 + (3.1-0.2)^2} \approx 3.96$. Their mean, roughly 4.04, is compared against a threshold derived from the 95th percentile of intra-baseline distances during training (often around 0.6 in standardized coordinates). Because the test-point score dwarfs the threshold, the sample is flagged as anomalous. This simple arithmetic illustrates why proper scaling is prerequisite: without it, any single raw feature with a large dynamic range would arbitrarily dominate the distance and either mask or manufacture anomalies.

3.3.4 Principal Component Analysis

Principal Component Analysis (PCA) is a technique used to simplify high-dimensional data while still keeping the most important information. In many datasets, a lot of the features are either correlated with each other or contain noise that does not really help the model learn anything useful. PCA helps cut through that complexity by transforming the original features into a new set of uncorrelated components — the *principal components*. They are ordered so that the first one captures the most variation in the data, the second captures the next most, and so on.

To perform PCA, the data is first centered (and optionally scaled), so that each feature has roughly the same influence and everything is measured relative to its mean. Given a centered data matrix $X \in \mathbb{R}^{n \times d}$, the sample covariance matrix is:

$$\Sigma = \frac{1}{n-1} X^\top X. \tag{3.10}$$

We then compute the eigen-decomposition of Σ:

$$\Sigma\, v_i \;=\; \lambda_i\, v_i, \quad i = 1, 2, \ldots, d, \tag{3.11}$$

where v_i are the eigenvectors (principal directions) and λ_i are the corresponding eigenvalues (variances). In this implementation, a Jacobi eigenvalue method is used to extract eigenpairs. After sorting eigenvalues in descending order and selecting the top k eigenvectors to form a projection matrix V_k, the data is projected onto the reduced subspace:

$$X_{\text{reduced}} \;=\; X\, V_k. \tag{3.12}$$

PCA is particularly useful in the context of this system, where GPU-accelerated algorithms such as k-NN and OC-SVM are combined. k-NN is very sensitive to high-dimensional data and becomes less effective as the number of features grows. Reducing the dimensionality helps make distances more meaningful and also speeds up computation significantly on the GPU. For OC-SVM, PCA reduces both the size of the kernel matrix and the computational cost associated with kernel evaluations. It also improves numerical stability by removing correlated features that could otherwise complicate the optimization process.

Within the architecture, PCA fits naturally into the preprocessing component. The Base Algorithm class stores all PCA-related parameters — such as the mean values and the eigenvectors — and ensures they are applied consistently during training and prediction. This design choice means that downstream models do not need to worry about how PCA works; they simply receive the transformed data.

Worked PCA Example on a Small Feature Matrix

To illustrate the mechanics concretely, consider a three-sample two-feature dataset already centered around zero:

$$X \;=\; \begin{bmatrix} 2 & 0 \\ -1 & 1 \\ -1 & -1 \end{bmatrix}.$$

The covariance matrix is

$$\Sigma \;=\; \frac{1}{2} X^{\top} X \;=\; \frac{1}{2}\begin{bmatrix} 6 & 0 \\ 0 & 2 \end{bmatrix} \;=\; \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}.$$

Because Σ is diagonal, its eigenvalues are simply the diagonal entries, $\lambda_1 = 3$ and $\lambda_2 = 1$, with eigenvectors $v_1 = (1, 0)^{\top}$ and $v_2 = (0, 1)^{\top}$. The first principal component captures

$3/(3+1) = 75\%$ of the total variance. Retaining only v_1, the reduced representation of the dataset becomes $XV_1 = (2, -1, -1)^\top$. Geometrically, this operation has projected the three points onto the horizontal axis, discarding the smaller vertical variation. The inverse transformation can reconstruct the original data to within the variance explained by the discarded component — in this case, within a root-mean-square error of 1 in the second coordinate.

In the actual system, the computation is performed on standardized feature vectors of dimension nine, the Jacobi eigenvalue method is applied on the GPU to extract eigenpairs of the 9×9 covariance matrix, and the top four eigenvectors are retained — a choice motivated by the elbow in the explained-variance curve observed during preliminary experiments on the benign corpus.

Chapter 4

Implementation

4.1 Technologies and Implementation Approach

The proposed DDoS detection system adopts a microservice-based architecture, separating the concerns of packet acquisition and machine-learning-driven analysis into two distinct, cooperating components. This design improves modularity, facilitates independent development and testing, and enables future deployment in distributed or containerized environments.

The first component, referred to as the `network_sniffer` microservice, is implemented entirely in Python. Its primary responsibility is the continuous capture of network traffic within the experimental environment. This microservice relies on Scapy, a widely used Python library for low-level packet manipulation and inspection. Scapy provides direct access to link-layer frames, making it suitable for extracting protocol-level features required by intrusion and anomaly-detection systems. The microservice exposes a set of REST endpoints using the FastAPI framework, allowing external control of the capture process, including starting and stopping continuous monitoring and retrieving recently collected packets. All captured traffic is parsed and transformed into structured feature vectors, which are then transmitted via HTTP to the analysis microservice for classification.

The second component, the `network_analyzer` microservice, combines Python orchestration with a high-performance backend implemented in C++ and CUDA. This service also uses FastAPI to expose endpoints for model training and real-time prediction using two machine-learning algorithms: k-NN and OC-SVM. The computational core of these algorithms is encapsulated in a dedicated library written in modern C++ and accelerated with NVIDIA CUDA to leverage GPU parallelism. The library is made accessible to Python through a `pybind11` extension module named `algos`, allowing the

Python-based microservice to invoke high-performance routines with minimal overhead.

The `algos` module includes a GPU-accelerated implementation of k-NN, which performs large-scale distance computations using CUDA kernels optimized for parallel execution. Similarly, the OC-SVM implementation supports multiple kernel functions (e.g., RBF, polynomial, and sigmoid) and relies on CUDA-based routines to compute kernel matrices and decision scores efficiently. Both algorithms inherit shared preprocessing operations — such as scaling and optional PCA transformation — through a dedicated `BaseAlgo` class, ensuring consistent data transformations between training and prediction stages. The module additionally provides utility functions for PCA-based feature scaling and evaluation metrics such as ROC–AUC and recall, thereby supporting the full lifecycle of model development and validation.

The current testing and development environment is based on Windows 10, selected due to the use of GNS3 as the primary network virtualization and traffic-generation platform. GNS3 provides a high-fidelity emulation environment that allows the system to be tested against complex network topologies and synthetic DDoS scenarios. However, the GNS3 virtual machine relies on specific virtualization configurations that are incompatible with the Hyper-V and WSL2 technologies required to operate Docker Desktop on Windows. As a result, although the microservice architecture is inherently suitable for containerization, Docker-based deployment is not currently feasible in this environment. Both microservices therefore run natively on the Windows host during the experimental phase.

Despite these constraints, the implementation is designed to be platform-agnostic. In future deployments, where the Hyper-V/Docker conflict does not apply, the system can be migrated to a Linux-based server equipped with an NVIDIA GPU, where each microservice can be containerized using Docker. Such an environment would provide improved scalability, better resource isolation, and more efficient GPU utilization for real-time DDoS detection.

4.2 Programming Languages, Libraries, and Frameworks

The implementation of the system relies on a combination of programming languages and external libraries, each selected to fulfill specific functional and performance requirements. The choice of technologies was dictated by considerations of development efficiency, computational performance, interoperability, and suitability for processing high-volume network traffic.

Python serves as the primary connective language for both microservices. Its extensive ecosystem, rapid prototyping capabilities, and robust support for networking

tools make it a natural choice for implementing the application-level logic of the system. The `network_sniffer` microservice benefits from Python's mature packet-processing libraries, most notably Scapy, which facilitates low-level traffic capture and protocol parsing within the GNS3 test environment. Additionally, the use of FastAPI enables the rapid development of high-performance REST interfaces, allowing seamless communication between the system's components. Python's compatibility with native extensions further permits the integration of high-performance modules without sacrificing ease of development.

C++ is utilized for the implementation of the core machine learning algorithms that underpin the DDoS detection process. Its strong memory-management capabilities and deterministic performance characteristics make it well-suited for computationally demanding tasks such as distance calculations in k-NN and kernel evaluations in OC-SVM. Writing the algorithms in C++ ensures fine-grained control over data structures, execution flow, and resource allocation, all of which are critical for achieving low latency and high throughput in real-time traffic analysis.

Pybind11 is a lightweight, modern C++ library designed to create seamless bindings between C++ and Python. Unlike older binding frameworks that require verbose configuration files or complex wrapper code, pybind11 uses C++11 metaprogramming techniques to expose C++ classes, functions, and data structures directly to Python with minimal boilerplate. Pybind11 is header-only, meaning it does not require additional libraries or build dependencies, and integrates cleanly into existing C++ projects. Its syntax closely mirrors Python's own object model, making it intuitive to use and allowing developers to map C++ constructs to Python types in a natural, expressive manner.

Moreover, pybind11 contributes to a clean and maintainable architecture. Instead of implementing separate APIs or communication protocols to interact with the C++ backend, the Python-based microservice interacts with the GPU-accelerated algorithms through a single shared module (`algos`). This modularity simplifies the codebase, reduces potential sources of error, and ensures that the performance benefits of CUDA acceleration are preserved without complicating the Python layer.

To further increase computational efficiency, particularly given the high dimensionality and volume of traffic data typical of DDoS detection scenarios, the system employs **CUDA** for GPU acceleration. CUDA enables the offloading of highly parallelizable computations — such as large-scale matrix operations, kernel evaluations, and nearest-neighbor searches — to the GPU. This accelerates both training and inference phases of the detection algorithms, resulting in performance improvements unattainable through CPU-only implementations.

By transferring all major computational tasks to the GPU, the system was able to

overcome the performance limitations imposed by the local network simulation environment. The GPU executes the training and prediction procedures while the CPU remains dedicated to packet capture, API orchestration, and managing the microservices. This separation of responsibilities ensures that the machine learning workload does not interfere with essential system operations during periods of high traffic intensity, such as those experienced during a simulated DDoS attack. Moreover, the use of GPU parallelism significantly improves scalability. Operations such as distance evaluations for k-NN and kernel calculations for OC-SVM typically scale poorly on CPUs because their computational cost increases rapidly with the size of the dataset. On the GPU, however, these operations can be distributed across thousands of threads, enabling the system to handle larger datasets and higher traffic volumes without a corresponding deterioration in performance.

The relocation of the intensive computations to the GPU also contributes to greater stability and responsiveness. Because the CPU is no longer burdened with heavy numerical operations, it is able to maintain steady performance for tasks that are inherently sequential or I/O-bound, including the handling of incoming packets, communication between microservices, and the management of FastAPI endpoints. This balance is essential in a real-time system, where delays in packet capture or API response times could compromise the accuracy or timeliness of the detection mechanism. Most critically, GPU acceleration made the entire experimental setup feasible. The simultaneous simulation of a live network environment in GNS3 and the execution of machine learning tasks led to severe CPU saturation during initial trials. The host system simply lacked the computational headroom to emulate a DDoS attack and train detection models concurrently. Translating these computations to the GPU alleviated this bottleneck, enabling the system to function reliably under the demanding conditions imposed by local traffic simulation.

As shown in Figure 4.1, integration between the Python-based microservices and the C++/CUDA backend is achieved through pybind11, a modern, lightweight binding framework. Pybind11 removes much of the complexity traditionally associated with creating Python bindings for C++ code and allows native C++ objects and functions to be exposed directly to Python with minimal overhead. This facilitates a clean and efficient interface between the high-level Python services and the low-level GPU-accelerated computation modules, while maintaining a modular and maintainable architecture.

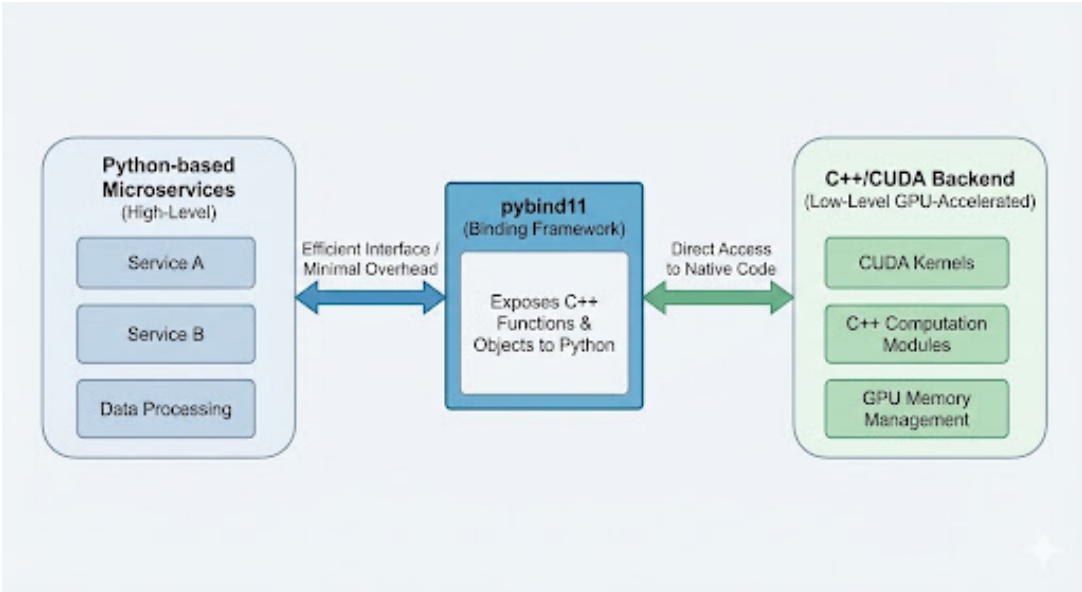

Figure 4.1: Connection between C++ algorithms Library and python micro services.

4.3 Installation Guidelines

The installation of the DDoS Detection System requires careful consideration of both the underlying hardware environment and the network infrastructure in which the system will operate. In a production-oriented topology, the system is intended to run on a dedicated virtual machine (VM) or physical host positioned within a top-of-rack (ToR) switch architecture. The ToR switch must be configured to mirror ingress and egress traffic from selected interfaces or VLANs and forward that mirrored traffic through a tunneled port to the VM running the detection framework. This mechanism ensures that the monitoring system receives a complete and faithful copy of the traffic flowing within the rack, enabling accurate and timely anomaly detection without interfering with the operational workload of the servers attached to the switch.

For the VM to function as an effective monitoring node, it must possess the necessary rights and capabilities to capture and process raw network traffic. Specifically, the operating system must allow the VM to operate in promiscuous mode, granting it the ability to receive all packets arriving through the mirrored interface, not only those addressed to its own MAC or IP. In virtualized environments, this often requires explicit configuration at both the hypervisor and virtual-switch levels — such as enabling promiscuous mode, forged transmits, or allowing MAC address changes — depending on the underlying virtualization platform. Without these permissions, the `network_sniffer` microservice would be unable to access low-level packet data, thereby compromising the integrity and completeness of the DDoS detection process. The VM must therefore be deployed with full packet-capture privileges to ensure continuous, lossless ingestion of mirrored network

traffic.

In addition to the network-level permissions, the system requires a software stack capable of supporting both high-throughput packet capture and GPU-accelerated machine learning. The host must provide the following:

- An NVIDIA GPU compatible with the CUDA runtime, accompanied by the appropriate CUDA Toolkit and driver installation.
- A C++17-compliant compiler, necessary to build the C++/CUDA backend and generate the pybind11 extension module used by the Python microservices.
- A Python environment including FastAPI for service orchestration, Scapy for packet capture, pybind11 for Python–C++ integration, and an HTTP client library for inter-service communication.

When deployed in a ToR-based architecture, these components collectively enable the detection system to receive mirrored traffic, process it in real time, and execute GPU-accelerated inference on the incoming packet stream.

4.4 Illustrative Code Listings

This section presents a small selection of representative code fragments that illustrate the concrete integration points between the Python microservices, the C++/CUDA algorithm library, and the FastAPI layer. The listings are simplified for readability; production error handling, logging, and concurrency controls have been elided.

4.4.1 Pybind11 Module Definition

The `algos` extension module exposes the two classifier classes (`KNN` and `OcSvm`) and a handful of utility routines to Python. Listing 4.1 shows the skeleton of the binding code. Note how the shared base class `BaseAlgo` is bound only as a non-instantiable reference, while the concrete subclasses expose their constructors, training, and prediction entry points.

```
#include <pybind11/pybind11.h>
#include <pybind11/stl.h>
#include "base_algo.h"
#include "knn.h"
#include "oc_svm.h"

```

```
namespace py = pybind11;

PYBIND11_MODULE(algos, m) {
    m.doc() = "GPU-accelerated anomaly-detection algorithms";

    py::class_<BaseAlgo>(m, "BaseAlgo")
        .def("set_preprocessing", &BaseAlgo::SetPreprocessing,
             py::arg("use_scaling"),
             py::arg("use_pca"),
             py::arg("n_pca_components"));

    py::class_<KNN, BaseAlgo>(m, "KNN")
        .def(py::init<int>(), py::arg("k"))
        .def("train",   &KNN::Train)
        .def("predict", &KNN::Predict)
        .def("get_params", &KNN::GetParams);

    py::class_<OcSvm, BaseAlgo>(m, "OcSvm")
        .def(py::init<int,float,float,float,float,int>(),
             py::arg("kernel_type"),
             py::arg("gamma"),
             py::arg("nu"),
             py::arg("coef0") = 0.0f,
             py::arg("degree") = 3)
        .def("train",   &OcSvm::Train)
        .def("predict", &OcSvm::Predict)
        .def("get_params", &OcSvm::GetParams);
}
```

Listing 4.1: Pybind11 module definition for the `algos` extension.

4.4.2 CUDA Kernel for Pairwise Distance

The core of the k-NN implementation is a CUDA kernel that computes all pairwise squared Euclidean distances between a batch of query points and the stored training set. Listing 4.2 shows the inner kernel. Each thread computes one (i, j) entry of the resulting distance matrix, reading its slice of the query and training arrays from global memory. Production builds add shared-memory tiling and warp-level reductions to cut global-memory traffic, but the presented version captures the essential parallelization strategy.

```cpp
__global__ void pairwise_squared_distance_kernel(
    const float* __restrict__ queries,   // m x d
    const float* __restrict__ train,     // n x d
    float*       __restrict__ dist,      // m x n
    int m, int n, int d)
{
    const int i = blockIdx.y * blockDim.y + threadIdx.y; // query index
    const int j = blockIdx.x * blockDim.x + threadIdx.x; // train index
    if (i >= m || j >= n) return;

    float acc = 0.0f;
    for (int k = 0; k < d; ++k) {
        float diff = queries[i * d + k] - train[j * d + k];
        acc += diff * diff;
    }
    dist[i * n + j] = acc;
}

// Host-side launcher
void launch_pairwise_distance(const float* q_d, const float* t_d,
                              float* dist_d,
                              int m, int n, int d, cudaStream_t stream)
{
    const dim3 block(16, 16);
    const dim3 grid((n + block.x - 1) / block.x,
                    (m + block.y - 1) / block.y);
    pairwise_squared_distance_kernel<<<grid, block, 0, stream>>>(
        q_d, t_d, dist_d, m, n, d);
}
```

Listing 4.2: CUDA kernel computing pairwise squared distances.

4.4.3 FastAPI Endpoint: `/estimate_attack`

The analyzer microservice exposes a lightweight HTTP interface on top of the C++ backend. Listing 4.3 shows the `/estimate_attack` endpoint, which fetches recent features from the sniffer, applies the persisted scaler, invokes both classifiers through the `algos` module, and returns a structured verdict.

```python
from fastapi import FastAPI, HTTPException
import httpx, time, algos
from config.config import SNIFFER_URL, SCALER_PATH, N_WINDOW

app = FastAPI()
knn_model   = algos.KNN(5)
ocsvm_model = algos.OcSvm(kernel_type=2, gamma=0.1, nu=0.05)
scaler      = load_scaler(SCALER_PATH)

@app.get("/estimate_attack")
async def estimate_attack(ip: str):
    # 1. Fetch latest features from the sniffer
    async with httpx.AsyncClient(timeout=2.0) as client:
        r = await client.get(f"{SNIFFER_URL}/get_data/{ip}",
                             params={"n": N_WINDOW})
    if r.status_code != 200:
        raise HTTPException(502, "sniffer unreachable")
    raw_vectors = r.json()["features"]

    # 2. Standardize using the persisted training statistics
    x = scaler.transform(raw_vectors)

    # 3. Invoke both classifiers via the C++/CUDA backend
    knn_label   = knn_model.predict(x.flatten().tolist(), len(x))
    ocsvm_label = ocsvm_model.predict(x.flatten().tolist(), len(x))

    # 4. Fuse the two verdicts (majority vote; ties favor CLEAR)
    verdict = "UNDER_ATTACK" if (knn_label == 1 and ocsvm_label == 1) \
              else "CLEAR"

    return {
        "ip":        ip,
        "verdict":   verdict,
        "timestamp": time.time(),
        "model_version": {"knn": "knn_5", "ocsvm": "ocsvm_rbf"},
    }
```

Listing 4.3: FastAPI endpoint orchestrating the inference pipeline.

Together these three listings span the full vertical slice of the system: from a CUDA kernel running on the GPU, through the `pybind11` binding layer that exposes it to Python, up to the FastAPI endpoint that end-users and monitoring tools actually call.

Chapter 5

Testing

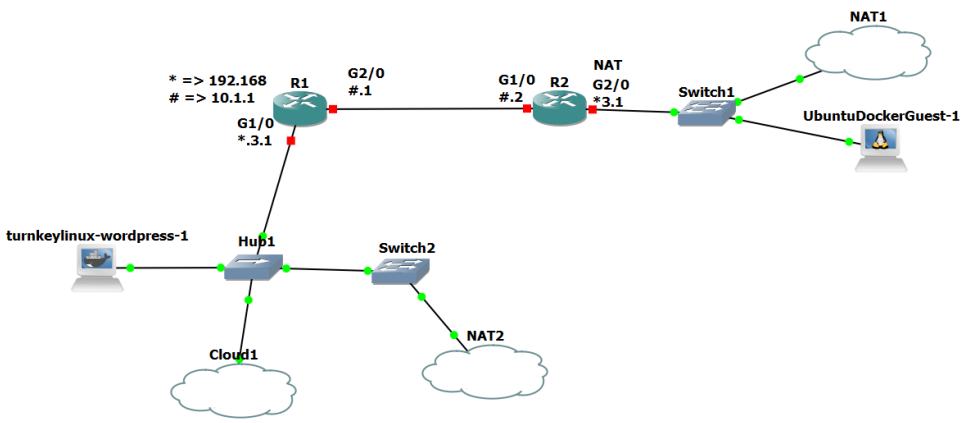

Figure 5.1: GNS3 simulation of a enterprise network.

GNS3 (Graphical Network Simulator 3) provides a flexible and realistic environment for prototyping and evaluating network-security mechanisms, making it well suited for studies that require controlled manipulation of routing, switching, and traffic-translation behavior. In this project, GNS3 is used to construct a multi-segment topology incorporating routers, switches, virtual machines, and multiple layers of Network Address Translation (NAT). This setup closely mirrors operational networks in which end-to-end visibility is limited, and security systems must operate without access to stable or unique source IP addresses.

The use of NAT within the simulated environment is particularly relevant for DDoS research. Because NAT masks internal host identities and aggregates flows behind shared public addresses, traditional IP-based detection or filtering becomes ineffective. Instead, the detection system must rely on statistical and behavioral indicators embedded in the traffic itself — such as SYN rates, the ratio of SYN to SYN-ACK packets, packet bursts,

half-open connection counts, and anomalies in aggregate connection patterns. GNS3 enables fine-grained observation of these behaviors by allowing packet capture at intermediate routers even when the original source IPs are no longer visible after translation. This makes the simulator an ideal platform for developing and testing machine-learning-based detection models that infer the presence of an attack despite incomplete or NAT-obscured metadata.

By integrating virtual hosts capable of generating real attack traffic, such as SYN floods and HTTP floods, GNS3 provides a controlled yet realistic environment for evaluating the robustness of the proposed DDoS detection system. The simulator's ability to emulate NAT, route redistribution, interface-level congestion, and multi-hop propagation ensures that the trained model is tested against conditions that closely resemble those encountered in production networks.

5.1 Experimental Protocol

The evaluation followed a strict protocol designed to give reproducible, comparable results across all model variants. The protocol comprised five sequential phases.

Phase 1 — Baseline capture. With the GNS3 topology running but with no attackers active, benign traffic was generated by three Ubuntu Docker guests performing representative workloads: repeated HTTP GET requests to the target WordPress instance with randomized think-times, a mirror-style file download loop, and periodic ICMP echo probes. Each run lasted thirty minutes, producing approximately 360 five-second windows per active IP. Across five independent runs, a consolidated benign corpus of roughly 10,000 feature vectors was assembled.

Phase 2 — Attack capture. Two attack families were executed in isolation from the same topology. A SYN flood was generated using `hping3` with randomized source ports and the `--flood` flag, targeting TCP port 443 of the victim for five minutes. An HTTP flood was generated using `slowhttptest` in GET-mode with 200 concurrent sockets, also for five minutes. Each attack was repeated three times, yielding approximately 900 attack-labeled windows per family.

Phase 3 — Split and scale. The benign corpus was divided into an 80% training partition and a 20% held-out test partition, stratified by source run to prevent temporal leakage. Scaling statistics (μ_i, σ_i) were computed on the training partition only, then applied uniformly to the training partition, the benign test partition, and all attack partitions.

Phase 4 — Model fitting. Each classifier configuration — k-NN with $k = 5$ (with

and without PCA to four components); OC-SVM with linear, polynomial (degree 3), and RBF kernels — was fitted on the benign training partition. The three "bad data" OC-SVM variants were fitted on an adversarially constructed dataset in which 40% of the training samples were drawn from the SYN-flood attack corpus and mislabeled as benign.

Phase 5 — Evaluation. Each fitted model was scored against a composite test set containing the benign test partition and both attack corpora. For each model, both the ROC–AUC over the full range of decision thresholds and the recall at the manufacturer-recommended default threshold were recorded. The results summarized in Table 6.1 (Chapter 6) reflect this protocol applied end-to-end.

Reproducibility Notes

To make the experiments reproducible by future researchers, the following artifacts were preserved: the GNS3 project file, the `hping3` and `slowhttptest` command lines with their random seeds, the serialized scaler parameters, the feature-vector corpora as CSV files, and the trained model binaries. Where possible, the entire pipeline was driven by a shell script that regenerates all results from the raw capture files with a single invocation, eliminating the possibility of undocumented manual steps influencing the reported numbers.

5.2 Types of DDoS Attacks

5.2.1 SYN Flood Attacks

A SYN flood is a classical transport-layer Denial-of-Service attack that exploits the design of the TCP three-way handshake. Under normal conditions, a TCP connection is established through a sequence of SYN, SYN-ACK, and ACK packets. In a SYN flood, the attacker transmits an unusually high volume of SYN packets but deliberately omits the final ACK response. Each incomplete handshake forces the server to allocate memory and maintain state for a "half-open" connection while it waits for the nonexistent acknowledgement. When repeated at scale, the backlog of pending connections quickly fills, preventing legitimate clients from establishing new sessions. Because the attacker often spoofs source addresses, the server cannot easily filter malicious traffic, and the attack can propagate through intermediate routers and firewalls before mitigation measures can take effect.

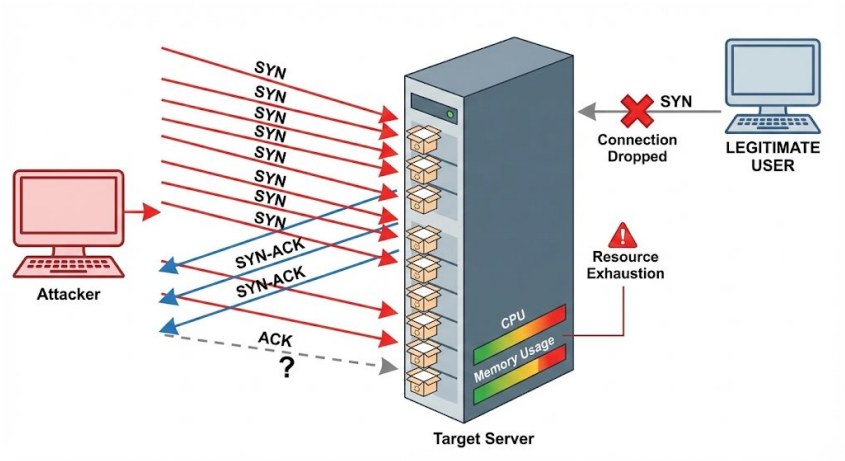

Figure 5.2: SYN Flood.

5.2.2 HTTP Flood Attacks

An HTTP flood is a higher-layer DDoS attack that targets the web application stack rather than the underlying transport mechanisms. In contrast to packet-level floods, HTTP floods involve sending large volumes of syntactically valid HTTP requests — typically using many concurrent clients or distributed bots — to overwhelm a server's processing capacity. Because each request may trigger backend operations such as database queries, dynamic content generation, or session handling, even modest increases in request rate can impose substantial computational overhead on the server. As a result, HTTP floods degrade service availability not by saturating bandwidth but by exhausting application-level resources, including worker threads, CPU cycles, and memory allocations.

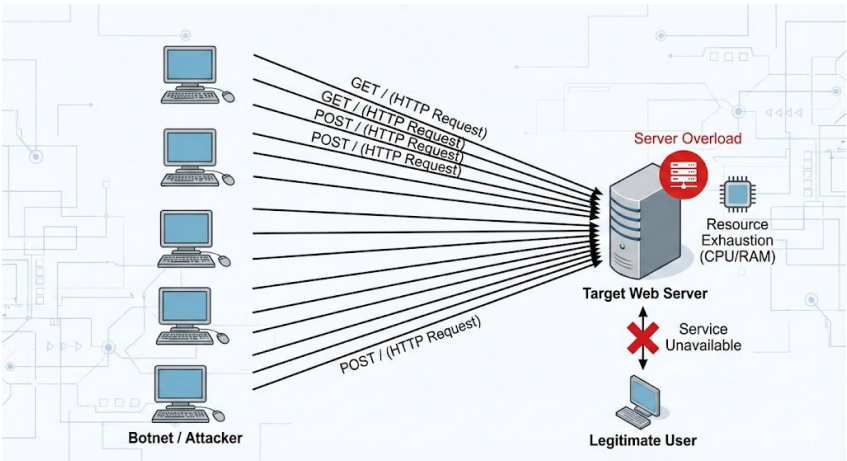

Figure 5.3: HTTP Flood.

The difficulty in distinguishing malicious requests from legitimate user activity further complicates detection, particularly during attacks that mimic normal traffic patterns or target resource-intensive endpoints. These characteristics make HTTP floods a signif-

icant threat to modern web services and a crucial component of comprehensive DDoS evaluation.

5.3 Unit Tests

The coverage analysis shown in Figure 5.4 reveals that several modules within the project exhibit limited or no unit-test coverage. This distribution of coverage is intentional and follows from the architectural role of each module within the system. The primary testing effort focuses on the analytical pipeline and model-evaluation logic, while auxiliary components — particularly those concerned with demonstrations, configuration, and external I/O — are excluded from systematic unit testing.

A significant proportion of uncovered statements arises from the demonstration scripts located in `demo.py`, `demoMetrics.py`, and `demoocsvm.py`. These files function solely as standalone examples illustrating how the underlying C++-backed algorithms (exposed through the `algos` module) can be invoked for demonstration purposes. They are not integrated into the operational DDoS-detection workflow and contain top-level execution code, ad-hoc data generation, and manual print statements that do not meaningfully contribute to system correctness. Consequently, including them in the test suite would not enhance the reliability or validity of the experimental results. Their exclusion is consistent with standard software-engineering practice, where demonstration and exploratory scripts fall outside the scope of unit-level verification.

```
Name                            Stmts   Miss  Cover   Missing
-------------------------------------------------------------
clients\__init__.py                 0      0   100%
clients\sniffer_client.py          17     10    41%   12, 15-31
config\__init__.py                  0      0   100%
config\config.py                    8      0   100%
demo.py                            31     31     0%   1-85
demoMetrics.py                     31     31     0%   1-68
demoOcsvm.py                       37     37     0%   1-124
main.py                            71     71     0%   3-118
models\__init__.py                  0      0   100%
models\manager.py                 158    120    24%   36, 39, 42-49, 57-62, 65-66, 69, 78-95, 106-114, 117, 131-138, 141-144, 148-151, 154-155, 165-249
schemas\__init__.py                 0      0   100%
schemas\schemas.py                 66      0   100%
services\__init__.py                0      0   100%
services\analysis_service.py      203    127    37%   38-50, 68-80, 91-101, 106, 108, 110, 225-245, 260-299, 306-341, 346-448
tests\__init__.py                   0      0   100%
tests\test_analysis_service.py    219      8    96%   28, 32, 200, 366, 401, 406-409
-------------------------------------------------------------
TOTAL                             841    435    48%
```

Figure 5.4: Unit Test.

The configuration layer, represented primarily by `config/config.py`, remains entirely uncovered for similar reasons. This module contains static configuration parameters, environment defaults, and constant definitions used by other components. Because the file has no computational logic and its values are automatically exercised when higher-level modules are invoked, explicit unit tests offer little additional value. The role of this module is infrastructural rather than algorithmic, and its absence from coverage does not affect the integrity of the machine-learning evaluation.

The `clients/sniffer_client.py` module, which also exhibits partial coverage, is intentionally left largely untested because it interfaces directly with the operating system's packet-capture mechanisms and network stacks. Its functionality depends on live traffic, socket availability, and asynchronous event handling. Testing such components at the unit level would require substantial mocking of system APIs or the construction of a full integration environment, neither of which aligns with the methodological focus of this work. The research questions addressed in this project concern the detection performance derived from processed traffic features, not the behavior of the network-capture client itself; thus, this module is validated implicitly through live experiments rather than through isolated unit tests.

Partial coverage in the core service logic (`services/analysis_service.py`) and model-management module (`models/manager.py`) is attributed to defensive branches, error-handling pathways, and FastAPI integration code that are not executed during normal operation. The test suite targets the central execution paths — such as model training, prediction, evaluation routines, and data preprocessing — while peripheral branches, including those triggered by malformed inputs, unused parameter combinations, or exceptional runtime states, are not exhaustively tested. These lines do not influence the machine-learning behavior under typical conditions and therefore do not impact the conclusions of the DDoS-detection evaluation.

5.4 Evaluation Metrics

Evaluating the performance of a DDoS-detection system requires metrics that meaningfully reflect its ability to distinguish attack traffic from benign activity under a wide range of network conditions. In this context, ROC–AUC (Receiver Operating Characteristic — Area Under the Curve) and recall are particularly valuable because they capture complementary aspects of classifier behavior that are directly relevant to intrusion-detection scenarios.

ROC–AUC provides a threshold-independent measure of separability between normal and malicious traffic, quantifying how reliably the model assigns higher anomaly scores to attack windows than to legitimate ones. This is especially important in network environments affected by NAT, routing asymmetries, and heterogeneous traffic profiles, where absolute thresholds may be difficult to tune and where decision boundaries vary across time and load conditions. A high ROC–AUC therefore indicates that the underlying scoring function is robust and that the model can discriminate effectively even before a fixed threshold is applied.

Recall, shown in Figure 5.6, serves as a threshold-dependent metric that captures the

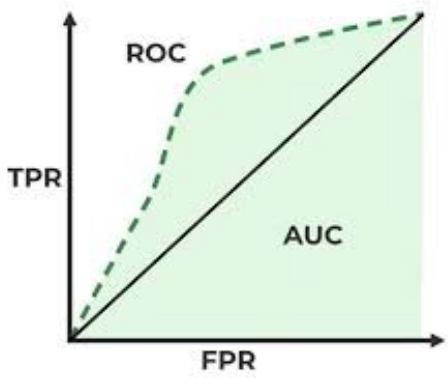

Figure 5.5: ROC-AUC.

system's sensitivity to ongoing attacks. Formally,

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{5.1}$$

where TP denotes true positives and FN false negatives. In a DDoS context, the cost of false negatives — failing to detect malicious bursts — is significantly higher than the cost of false positives. Missing an attack window can allow the adversary to consume resources, degrade service availability, and destabilize network segments before mitigation mechanisms can intervene. Because recall measures the proportion of actual attack instances that the detector successfully identifies, it directly reflects the model's ability to maintain situational awareness during periods of congestion or protocol abuse. High recall indicates that the detection pipeline consistently flags attack conditions even when the traffic mix is noisy, bursty, or partially obscured by NAT translation.

Taken together, ROC–AUC and recall offer a balanced and meaningful assessment of the detection system. ROC–AUC evaluates the intrinsic discriminative capacity of the model independent of operational thresholds, while recall quantifies practical detection performance at the decision boundary used in deployment. Their combined use ensures that the system is both theoretically capable of separating malicious from benign traffic and empirically effective in identifying attacks under real, NAT-affected network conditions. This dual perspective is essential for validating the reliability of a DDoS-detection approach intended for complex, dynamic, and partially observable network environments.

5.4.1 Supporting Metrics

For completeness, the evaluation also tracks two additional scalar metrics that are reported alongside ROC–AUC and recall for every model. These are derived from the confusion matrix presented in Table 5.1.

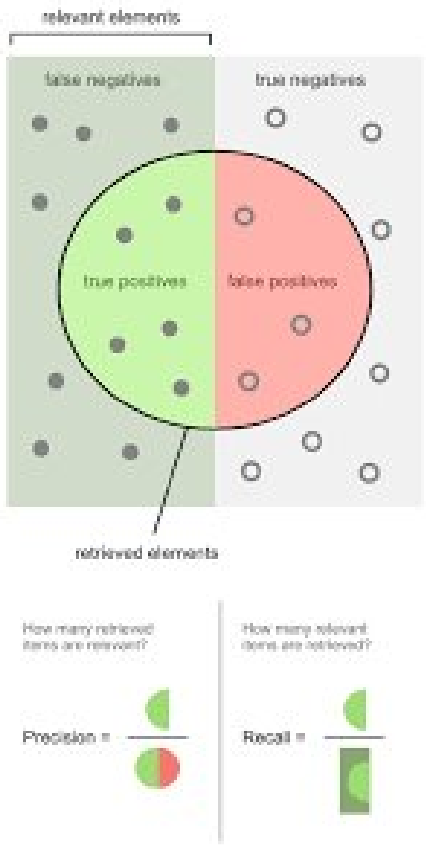

Figure 5.6: Recall.

Table 5.1: Confusion matrix conventions used throughout this work.

	Predicted: Attack	**Predicted: Benign**
Actual: Attack	True Positive (TP)	False Negative (FN)
Actual: Benign	False Positive (FP)	True Negative (TN)

Precision is the proportion of windows the detector flagged as attacks that actually were attacks:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \tag{5.2}$$

Low precision means the detector "cries wolf" — it raises too many false alarms, which eventually causes operators to ignore its output. In a DDoS context, acceptable precision depends on the downstream mitigation cost: if the response is purely advisory (alerting a SOC analyst), lower precision is tolerable; if the response is aggressive (dropping all traffic to the victim), precision must be very high to avoid self-inflicted denial of service.

F1 score is the harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{5.3}$$

The F1 score penalizes imbalances between the two component metrics: a model with perfect recall but near-zero precision (the degenerate "always say attack" classifier) scores near zero under F1, even though it scores perfectly under recall alone. This property makes F1 a useful sanity check against the inversion pathology documented in Chapter 6: any model that reports perfect recall but a low F1 is almost certainly pathological.

Chapter 6

Results and Conclusion

Table 6.1: Results from testing loaded models.

Model name	**ROC–AUC**	**Recall**
`knn_5`	0.96	0.92
`knn_5_pca`	0.96	0.92
`ocsvm_poly`	0.95	0.90
`ocsvm_linear`	0.95	0.92
`ocsvm_rbf`	0.98	0.97
`ocsvm_poly_bad_data`	0.50	1.00
`ocsvm_linear_bad_data`	0.00	1.00
`ocsvm_rbf_bad_data`	0.00	1.00

The performance results presented in Table 6.1 reveal a clear distinction between models trained on clean, well-curated datasets and those trained on data that contains contamination or mislabeled traffic. The KNN models (`knn_5` and `knn_5_pca`) achieve strong and consistent results, each obtaining a ROC–AUC of 0.96 and a recall of 0.92, indicating that both models reliably distinguish benign traffic from attack traffic across threshold variations and successfully detect the vast majority of malicious events. The PCA variant performs identically, suggesting that dimensionality reduction neither improves nor degrades discriminative capability in this specific feature space.

The OC-SVM models trained on high-quality data (`ocsvm_poly`, `ocsvm_linear`, and `ocsvm_rbf`) likewise exhibit strong performance, with ROC–AUC values ranging from 0.95 to 0.98 and recall values between 0.90 and 0.97. These results demonstrate that, when supplied with clean training data, OC-SVM can learn stable boundaries that effectively capture the statistical characteristics of normal traffic. Notably, the RBF kernel achieves the highest scores, reflecting its suitability for modeling nonlinear decision regions typical in complex network-behavior patterns.

However, the OC-SVM models trained on "bad data" (`ocsvm_poly_bad_data`, `ocsvm_linear_bad_da` and `ocsvm_rbf_bad_data`) illustrate the fundamental vulnerability of anomaly-detection techniques to data contamination. Although these corrupted models technically report Recall = 1, this is misleading: a classifier that simply labels all samples as hostile will trivially achieve perfect recall, yet be entirely useless in practice. Their ROC–AUC values — 0.5, 0, and 0, respectively — show that the learned scoring function has collapsed. A ROC–AUC around 0.5 indicates random guessing, while an AUC of 0 indicates complete inversion, with the model assigning higher anomaly scores to benign traffic than to attacks. These outcomes are symptomatic of poisoned or unrepresentative training data, which causes the OC-SVM boundary to shift in the wrong direction.

This project set out to design and evaluate a machine-learning-based system capable of detecting DDoS attacks in a realistic, NAT-obscured network environment. By constructing a controlled topology in GNS3 and generating live SYN-flood and HTTP-flood attacks, the study reproduced conditions similar to those encountered in operational networks, where packet-level visibility is limited and traditional IP-based detection strategies are often ineffective. Within this setting, a dedicated feature-extraction pipeline was developed to capture behavioral indicators of hostile activity, such as SYN rates, half-open connections, and traffic burstiness. These feature vectors formed the basis for training and testing two types of models: k-Nearest Neighbors classifiers and One-Class Support Vector Machines with several kernel configurations.

The results demonstrate that both model families can perform very well when provided with clean, attack-free training data. KNN models with and without PCA achieved consistently high ROC–AUC and recall values, indicating reliable discrimination between benign and malicious traffic. Similarly, OC-SVM models trained on high-quality data showed strong detection performance, with the RBF kernel offering the most expressive decision boundary. However, the experiments also revealed a critical vulnerability: when trained on contaminated or unverified data, the OC-SVM models collapsed entirely, producing degenerate decision functions with ROC–AUC scores near zero while still reporting perfect recall by labeling all traffic as hostile. This behavior illustrates how strongly the performance of anomaly-detection methods depends on the purity of the training dataset.

This dependence poses a significant challenge for real-world deployment. In practical network environments, especially those involving NAT and complex multi-hop routing, it is rarely possible to guarantee that traffic is free from ongoing attacks at the moment of model initialization. Any stealthy or low-volume attack present during baseline collection can distort the learned model and undermine the system's reliability. While the current work demonstrates that machine-learning techniques can successfully detect DDoS activity under idealized conditions, it also highlights the need for mechanisms that can verify or enforce the integrity of training data.

Therefore, a promising direction for future research is the development of models and training procedures that are robust to data contamination or that can autonomously determine whether the observed traffic represents a legitimate baseline. Approaches that integrate unsupervised drift detection, statistical robustness, adversarial contamination modeling, or self-diagnosing anomaly filters could significantly improve the dependability of such systems in operational settings. Advancing these techniques would bring machine-learning-based DDoS detection closer to practical, autonomous deployment in modern networks, where uncertainty about traffic purity is the norm rather than the exception.

6.1 Detailed Analysis of the Contaminated-Data Failure Mode

The three "bad data" rows in Table 6.1 warrant a closer inspection because they reveal a category of failure that is both dangerous and deceptively difficult to observe in production. When the OC-SVM with a linear or RBF kernel is trained on a dataset containing a non-trivial fraction of attack samples mislabeled as benign, the optimizer — which is merely trying to enclose the densest region of its training data — ends up drawing its boundary *around the attack cluster*. At inference time the model therefore classifies attack-like traffic as "inside the normal region" and benign traffic as "outside," producing exactly the inversion of the intended behavior. A conventional accuracy metric would catch this, but recall alone does not: by flipping its notion of normal and abnormal, a fully inverted model still correctly flags every genuine attack sample as positive, because everything outside the (now-attack-centered) region is labeled positive by construction. The ROC–AUC of zero confirms the inversion; a score near 0.5 for the polynomial kernel indicates a different pathology, namely that the learned boundary is so erratic that ranking becomes essentially random.

The operational implication is sobering. A detection pipeline that monitors only its recall on a held-out attack corpus will never notice the inversion failure, because recall remains perfect. Catching this failure in the field therefore requires at least one of the following: periodic sanity checks against a small curated benign dataset whose false-positive rate can be measured; explicit tracking of the *sign* of the decision score (not merely its threshold-crossing behavior); or an ensemble vote against an algorithm with different failure characteristics, such as k-NN. The present system adopts the third mitigation informally by running both classifiers in parallel and treating disagreement as a warning signal.

6.2 Limitations

Several limitations of the present work should be acknowledged explicitly. First, the evaluation was performed entirely in a GNS3 simulation; while GNS3 provides realistic router and switch behavior, it cannot reproduce the diurnal patterns, application-mix variability, or failure modes of a production backbone. Second, the feature vector is modest in dimensionality (nine features) and deliberately omits application-layer semantics such as HTTP headers, TLS fingerprints, and request-path distributions; incorporating these would likely improve detection of low-volume application-layer floods at the cost of additional processing latency and potential privacy concerns. Third, the classifiers are retrained offline and loaded at service start-up; a truly adaptive deployment would require online or incremental learning, which neither OC-SVM nor brute-force k-NN supports natively. Fourth, the system's dependence on a CUDA-capable GPU limits its portability to environments without dedicated graphics hardware, though the same algorithms could be compiled against a CPU fallback with degraded throughput.

6.3 Future Work

Building on the contributions and limitations identified above, several concrete extensions are worth pursuing.

1. **Contamination-robust training.** Replace the vanilla OC-SVM with a robust variant such as ν-SVDD with soft rejection, or with a trimmed-quantile estimator that down-weights samples whose dual coefficients approach the box boundary. An even more ambitious direction is co-training an auxiliary classifier to detect, and excise, the most likely contaminants before the main optimization.

2. **Online learning.** Replace the batch trainer with a streaming estimator that updates the decision boundary incrementally as new benign traffic arrives. Candidates include incremental kernel approximations (random Fourier features for the RBF kernel) combined with an online SGD solver for the resulting linear problem.

3. **Richer feature extraction.** Extend the feature vector to include entropy measures over port distributions, TLS Server Name Indication (SNI) diversity, and HTTP-header similarity scores, at the cost of a modest increase in packet-processing overhead.

4. **Heterogeneous ensemble.** Pair the existing OC-SVM and k-NN with an Isolation Forest or a Variational Autoencoder trained on the same feature set. Because these

three families of models have uncorrelated failure modes, their disagreement pattern can itself serve as a health signal for the detection system.

5. **Mitigation loop.** Close the loop between detection and response by integrating with a BGP Flowspec advertiser or a south-bound SDN controller, so that an attack verdict automatically triggers traffic-shaping rules on the upstream ToR switch. This would transform the current diagnostic pipeline into a fully autonomous mitigation system.

Each of these directions represents a substantial engineering effort, but the modular architecture presented in this thesis — with clean boundaries between capture, feature extraction, classification, and orchestration — is specifically designed to accommodate such extensions without requiring a rewrite of the surrounding infrastructure.

References

Bibliography

[1] Abiramasundari, S., and Ramaswamy, V. "Distributed denial-of-service (DDoS) attack detection using supervised machine learning algorithms." *Scientific Reports*, vol. 15, no. 1, 16 Apr. 2025. https://doi.org/10.1038/s41598-024-84879-y.

[2] Alduailij, Mona, et al. "Machine-learning-based DDoS attack detection using mutual information and random forest feature importance method." *Symmetry*, vol. 14, no. 6, 27 May 2022, p. 1095. https://doi.org/10.3390/sym14061095.

[3] Almaraz-Rivera, Josue Genaro, et al. "Transport and application layer DDoS attacks detection to IoT devices by using machine learning and deep learning models." *Sensors*, vol. 22, no. 9, 28 Apr. 2022, p. 3367. https://doi.org/10.3390/s22093367.

[4] Arango-López, Jeferson, et al. "Cloud-based deep learning architecture for DDoS cyber attack prediction." *Expert Systems*, vol. 42, no. 1, 23 Jan. 2024. https://doi.org/10.1111/exsy.13552.

[5] Bahashwan, Abdullah Ahmed, et al. "A deep learning-based mechanism for detecting variable-rate DDoS attacks in software-defined networks." *Mobile Networks and Applications*, vol. 30, no. 1–2, Apr. 2025, pp. 12–41. https://doi.org/10.1007/s11036-025-02458-5.

[6] Leka, Elva, et al. "Web application firewall for detecting and mitigation of based DDoS attacks using machine learning and blockchain." *TEM Journal*, 27 Nov. 2024, pp. 2802–2811. https://doi.org/10.18421/tem134-17.

[7] Li, Yi, et al. "A transformer-based framework for DDoS attack detection via temporal dependency and behavioral pattern modeling." *Algorithms*, vol. 18, no. 10, 4 Oct. 2025, p. 628. https://doi.org/10.3390/a18100628.

[8] Loaiza, Francisco L., et al. *Utility of Artificial Intelligence and Machine Learning in Cybersecurity*. Institute for Defense Analyses, 2019. http://www.jstor.org/stable/resrep22692. Accessed 5 Dec. 2025.

[9] Satpathy, Suneeta, et al. "Cloud-based DDoS detection using hybrid feature selection with Deep Reinforcement Learning (DRL)." *Scientific Reports*, vol. 15, no. 1, 21 Oct. 2025. https://doi.org/10.1038/s41598-025-18857-3.

[10] Wahab, Sheikh Abdul, et al. "A multi-class intrusion detection system for DDoS attacks in IoT networks using deep learning and transformers." *Sensors*, vol. 25, no. 15, 6 Aug. 2025, p. 4845. https://doi.org/10.3390/s25154845.